

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



INF3812 – Deep Learning

Entrenamiento de Redes Neuronales

Hans Löbel

Dpto. Ingeniería de Transporte y Logística
Dpto. Ciencia de la Computación

En teoría, no hay diferencia entre la teoría y la práctica. En la práctica, sí la hay.

Tras bambalinas, hacer que los modelos de DL funcionen adecuadamente, y sean prácticos, requiere una gran cantidad de detalles:

- ¿Cómo optimizar?
- ¿Cómo controlar el sobreajuste?
- ¿Cómo reutilizar?

En teoría, no hay diferencia entre la teoría y la práctica. En la práctica, sí la hay.

Tras bambalinas, hacer que los modelos de DL funcionen adecuadamente, y sean prácticos, requiere una gran cantidad de detalles:

- ¿Cómo optimizar?
- ¿Cómo controlar el sobreajuste?
- ¿Cómo reutilizar?

Recordemos primeramente qué estamos haciendo al entrenar una red neuronal

Problema de optimización: búsqueda sobre el espacio de hipótesis

Parámetros: viven en el espacio de hipótesis del modelo

Función de pérdida: captura el rendimiento del modelo al cuantificar su error en los datos de entrenamiento.

Etiqueta: valor a predecir (tarea)

$$\operatorname{argmin}_W J(X, Y; W) = \lambda \mathcal{R}(W) + \sum_i^N \mathcal{L}(f(x_i; W), y_i)$$

Datos de entrenamiento: incluyen los datos en sí y sus etiquetas.

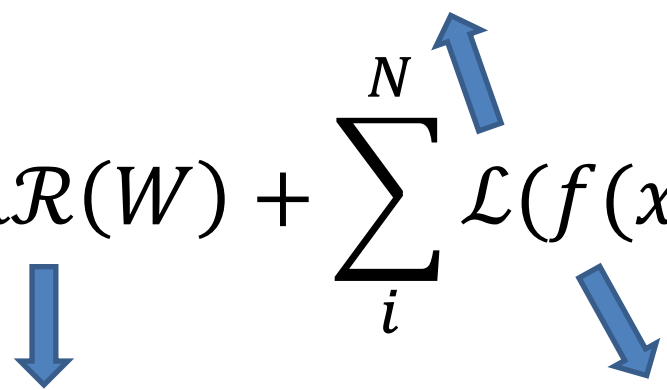
Regularizador: función que induce sesgo inductivo en el modelo predictivo a través de sus parámetros.

Modelo predictivo: función parametrizada, que mapea desde el espacio de los datos hacia el de las etiquetas

Buscamos, en el **espacio de hipótesis** del modelo, aquella con mejor rendimiento (**menor pérdida**) en los datos de **entrenamiento**, apoyándonos en el **sesgo inductivo** del modelo.

Recordemos primeramente qué estamos haciendo al entrenar una red neuronal

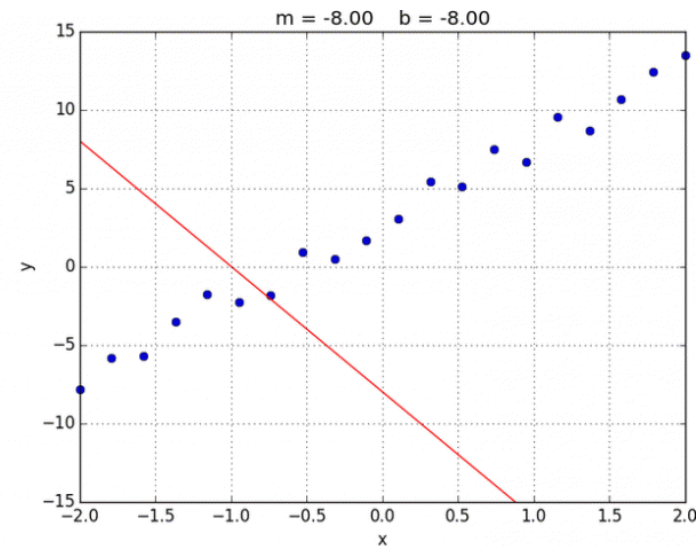
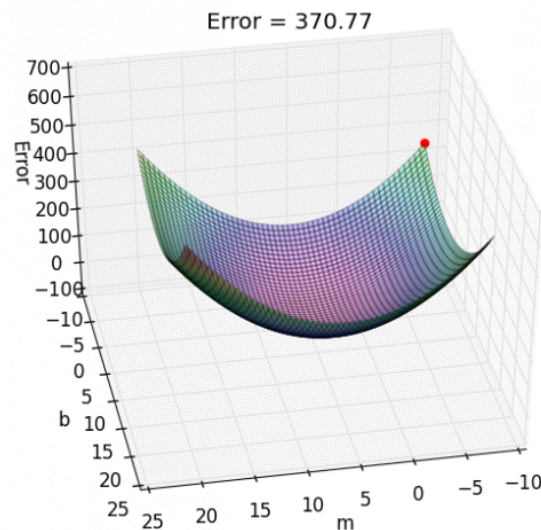
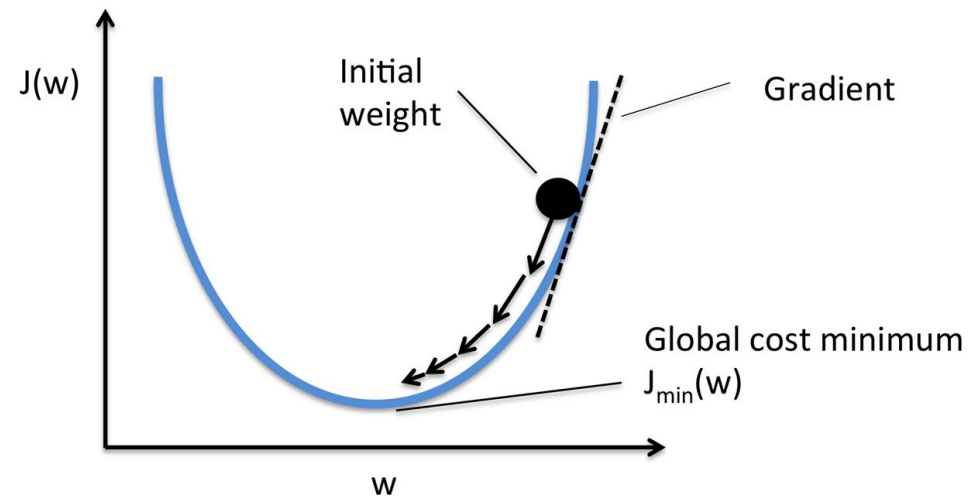
Función de pérdida: captura el rendimiento del modelo al cuantificar su error en los datos de entrenamiento.

$$\operatorname{argmin}_W J(X, Y; W) = \lambda \mathcal{R}(W) + \sum_i^N \mathcal{L}(f(x_i; W), y_i)$$


Regularizador: función que induce sesgo inductivo en el modelo predictivo a través de sus parámetros.

Modelo predictivo: función parametrizada, que mapea desde el espacio de los datos hacia el de las etiquetas.

Por ahora, tomaremos un enfoque de descenso de gradiente clásico para la optimización



Dado el enfoque de descenso, surgen inmediatamente múltiples preguntas relevantes

Para obtener los parámetros de una red a través de un proceso de optimización, debemos considerar bastantes elementos:

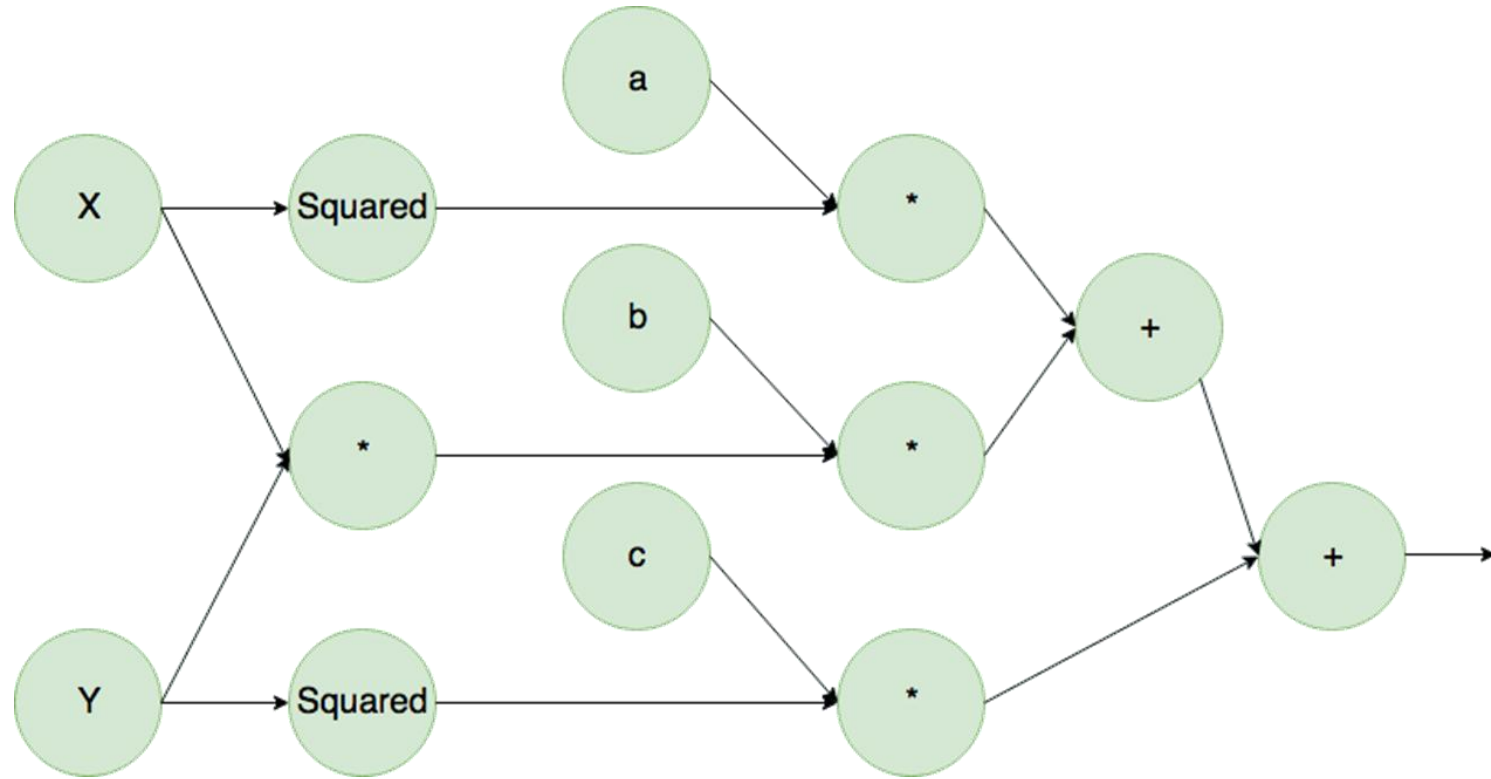
- ¿Cómo calculo las derivadas para cada parámetro?
- ¿Cómo descendo de manera efectiva y eficiente en la función objetivo?
- ¿Debo inicializar los pesos y normalizar los datos de manera especial?
- ¿Puedo hacerle algo a la arquitectura de la red para facilitar el proceso?

Dado el enfoque de descenso, surgen inmediatamente múltiples preguntas relevantes

Para obtener los parámetros de una red a través de un proceso de optimización, debemos considerar bastantes elementos:

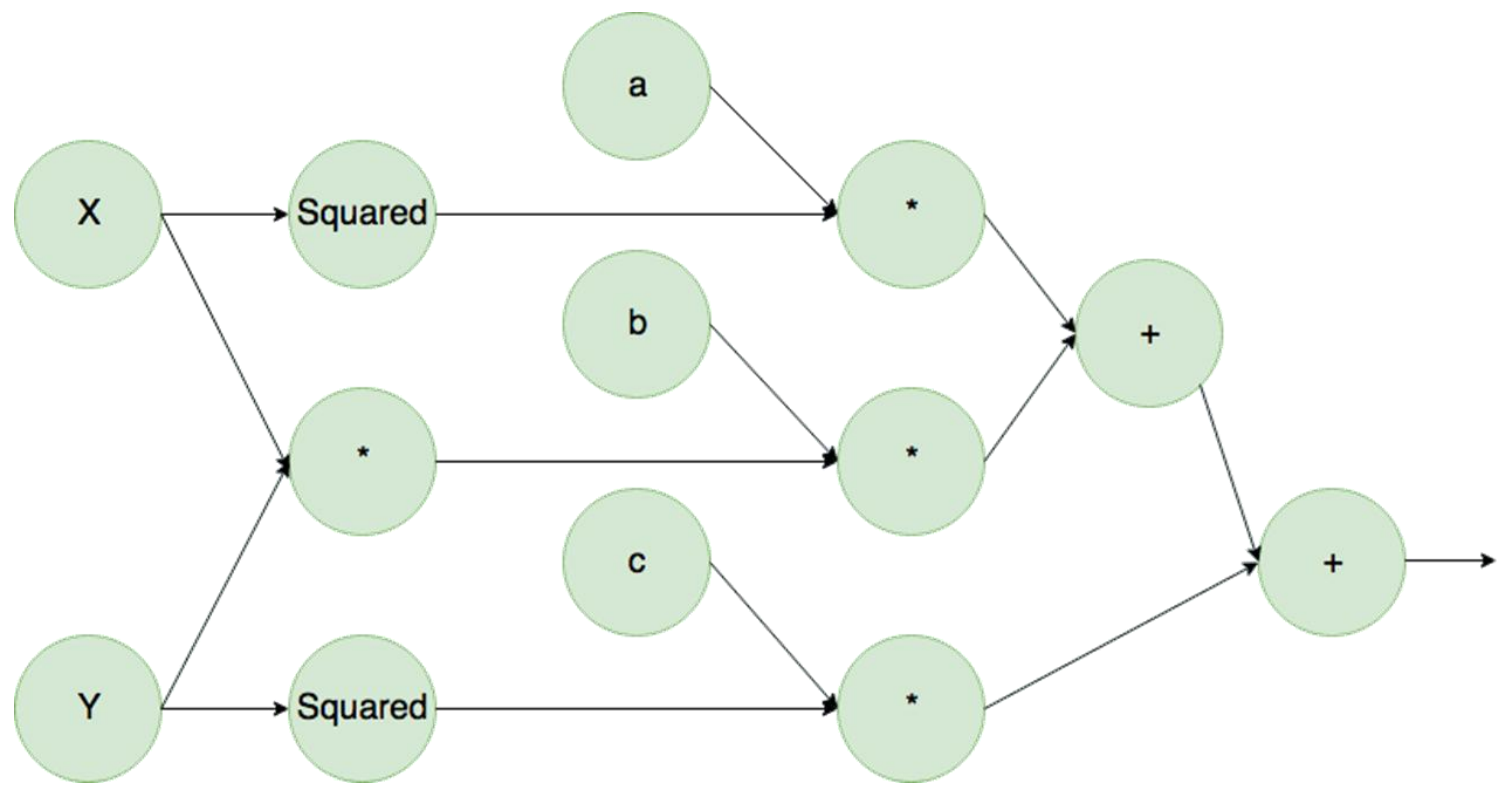
- ¿Cómo calculo las derivadas para cada parámetro?  $\frac{\partial \mathcal{L}}{\partial w}, \forall w \in W$
- ¿Cómo descendo de manera efectiva y eficiente en la función objetivo?
- ¿Debo inicializar los pesos y normalizar los datos de manera especial?
- ¿Puedo hacerle algo a la arquitectura de la red para facilitar el proceso?

Un grafo de cómputo representa **todas las operaciones** realizadas hasta llegar al output

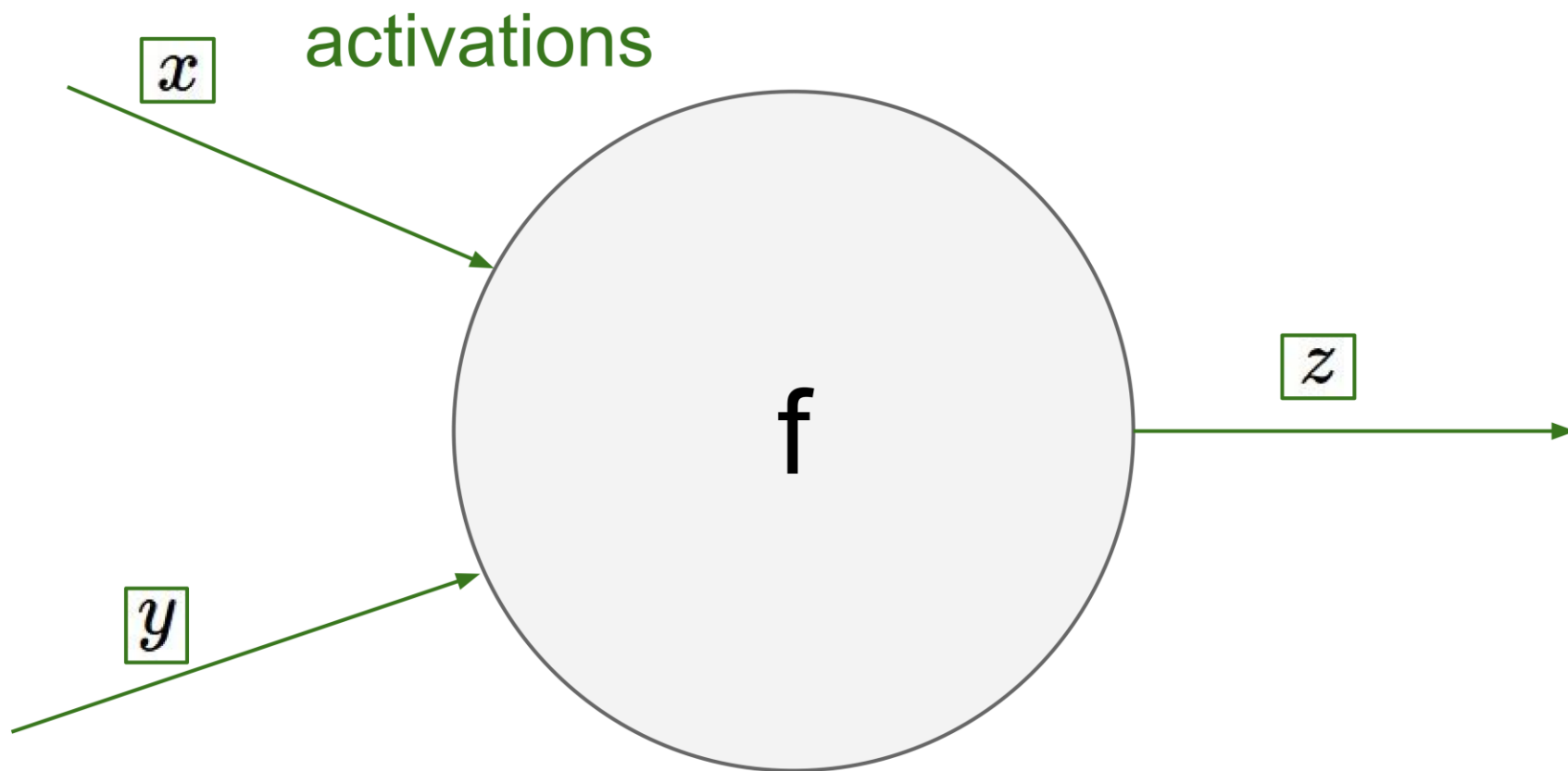


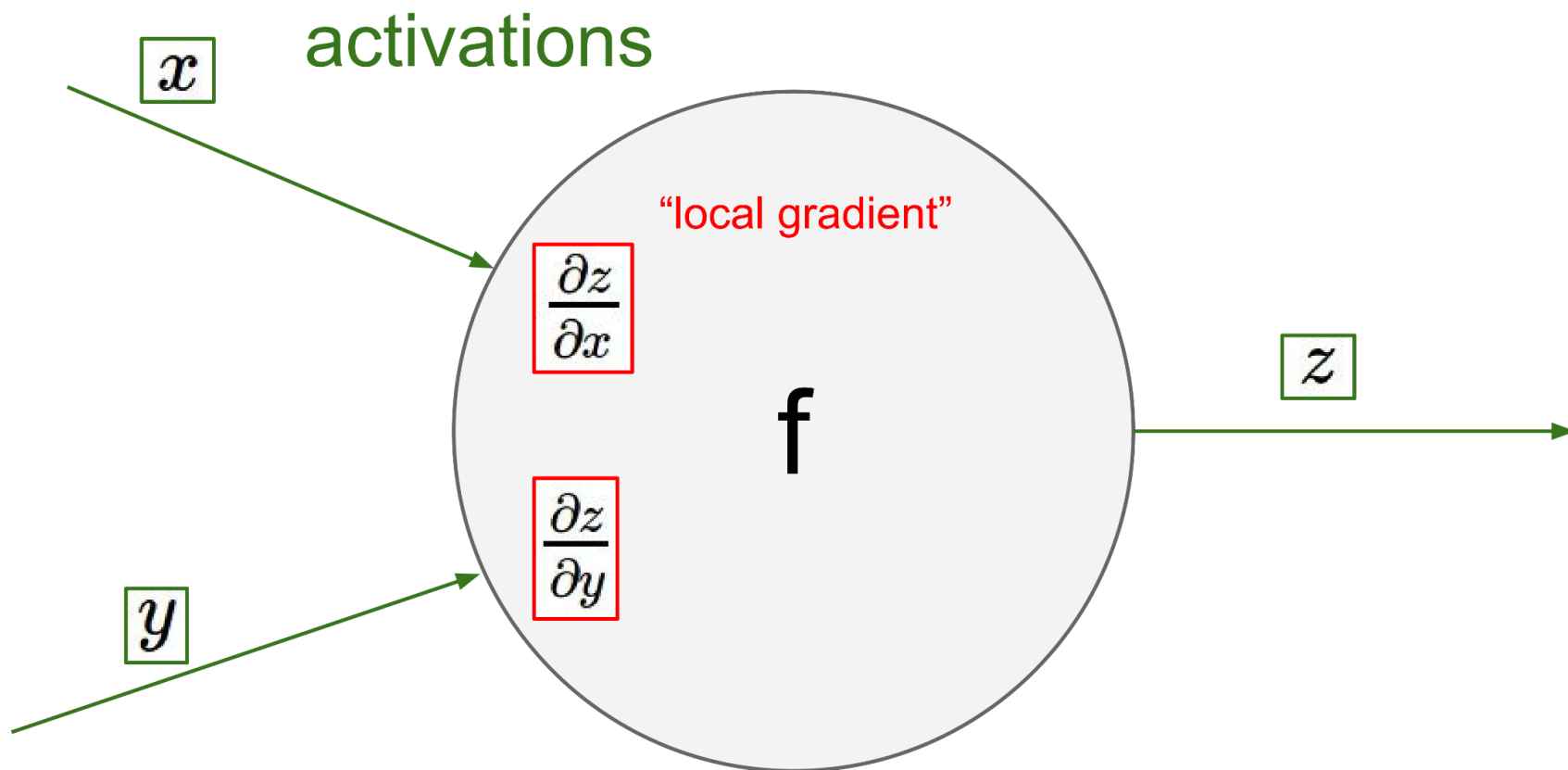
¿Qué función representa este grafo?

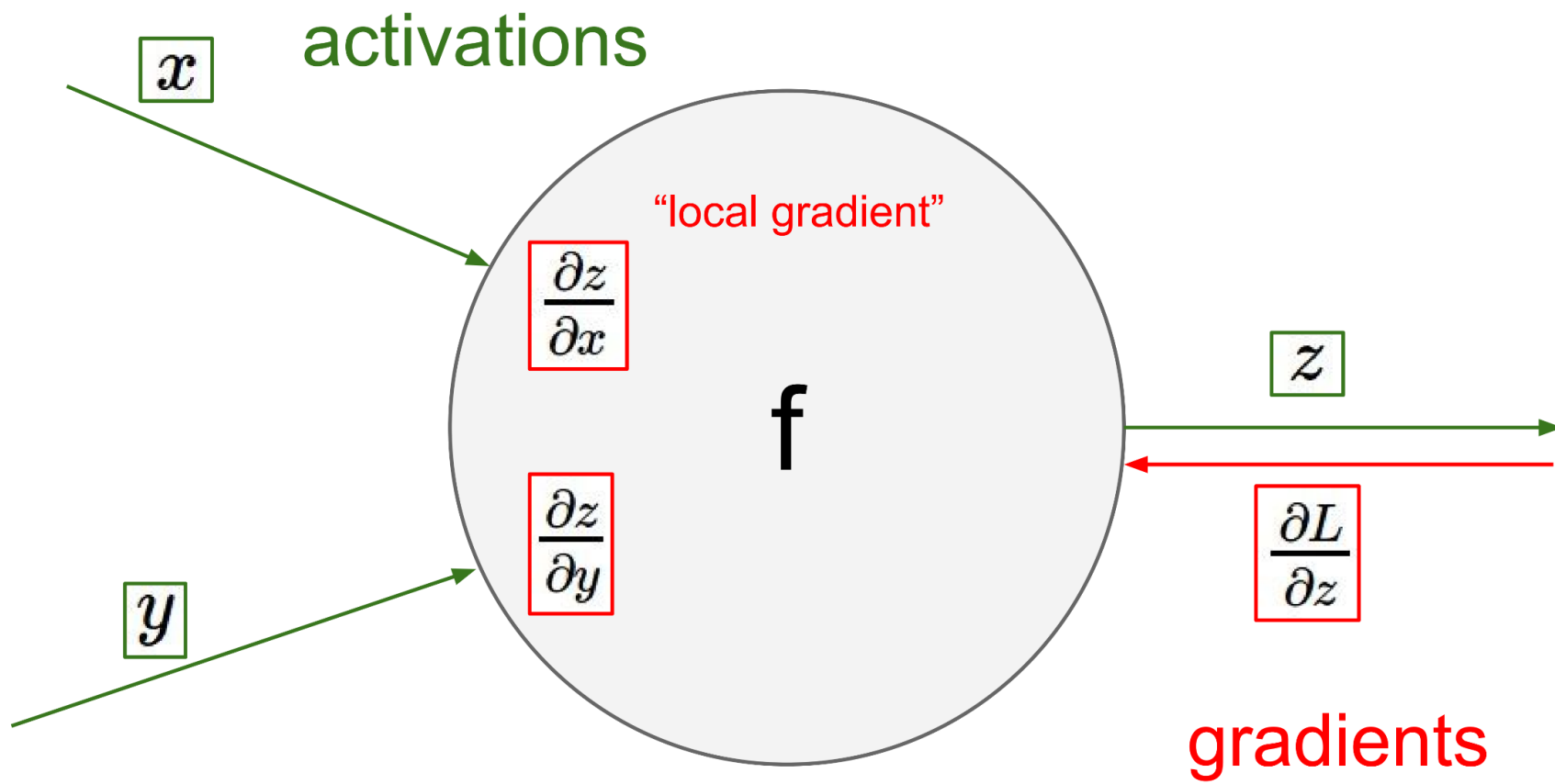
Podemos aprovechar esta **estructura** junto con la regla de la cadena, para obtener el gradiente completo

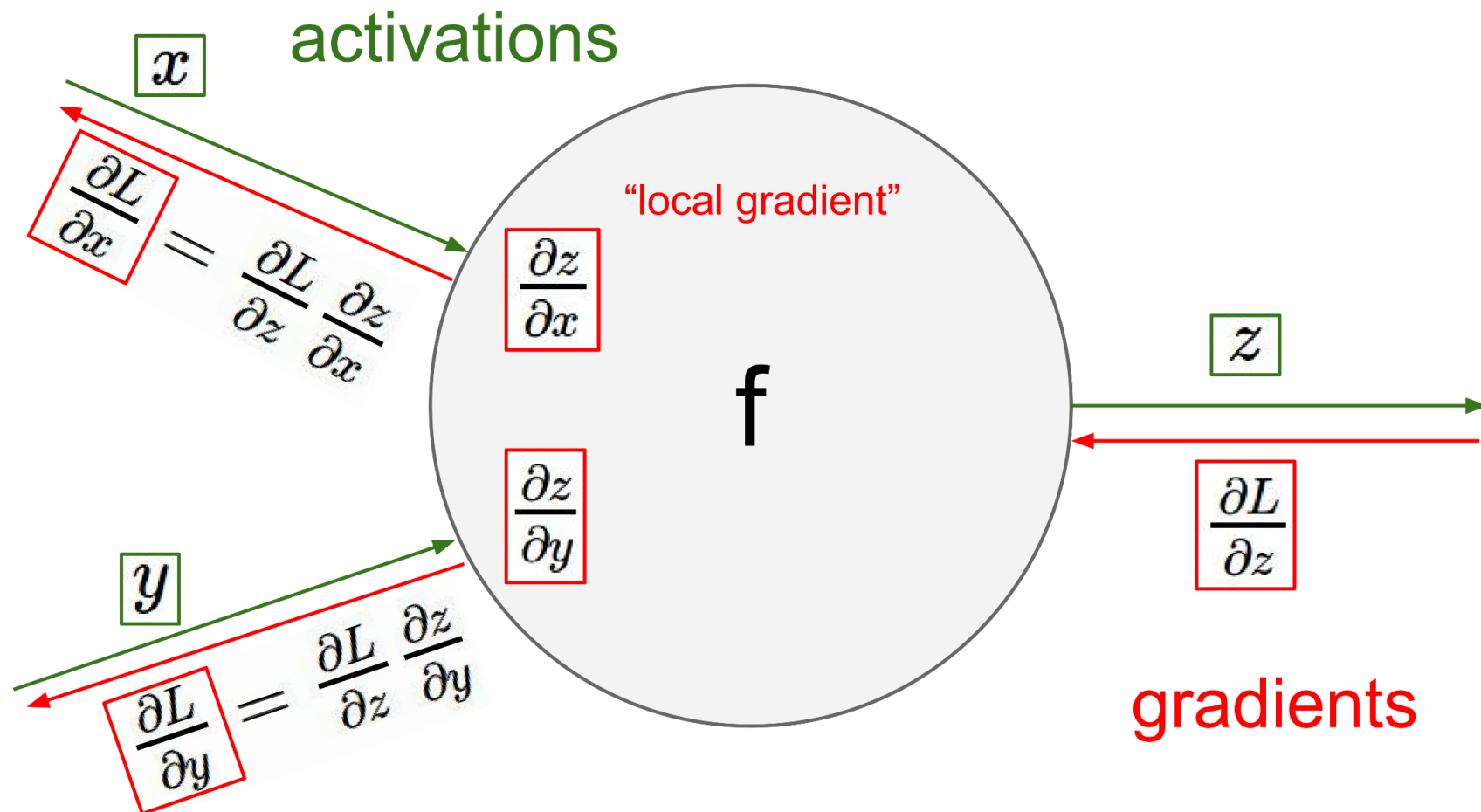


Basta con que solo analicemos
las derivadas locales

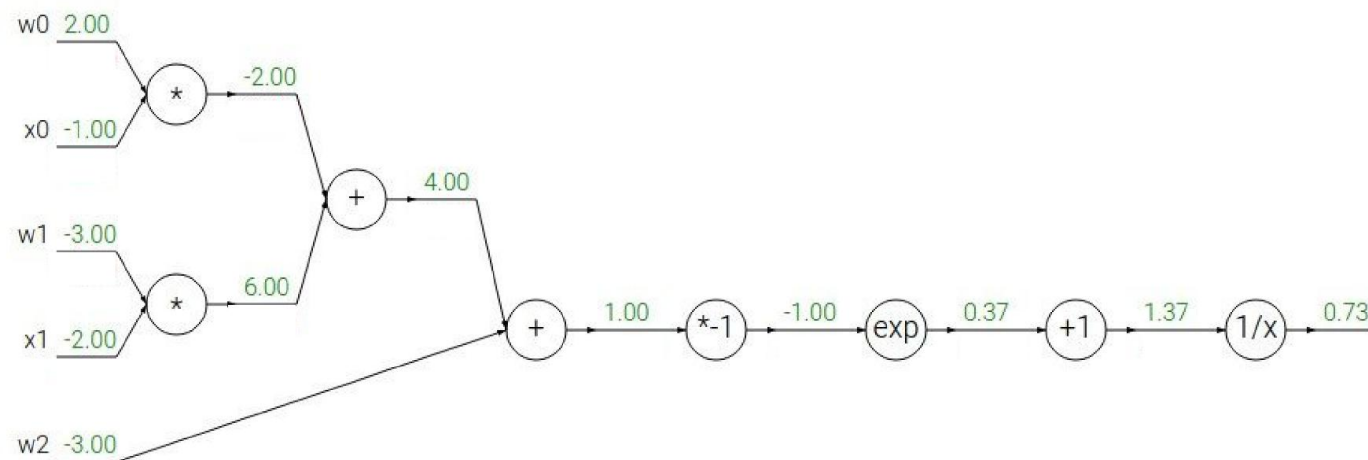




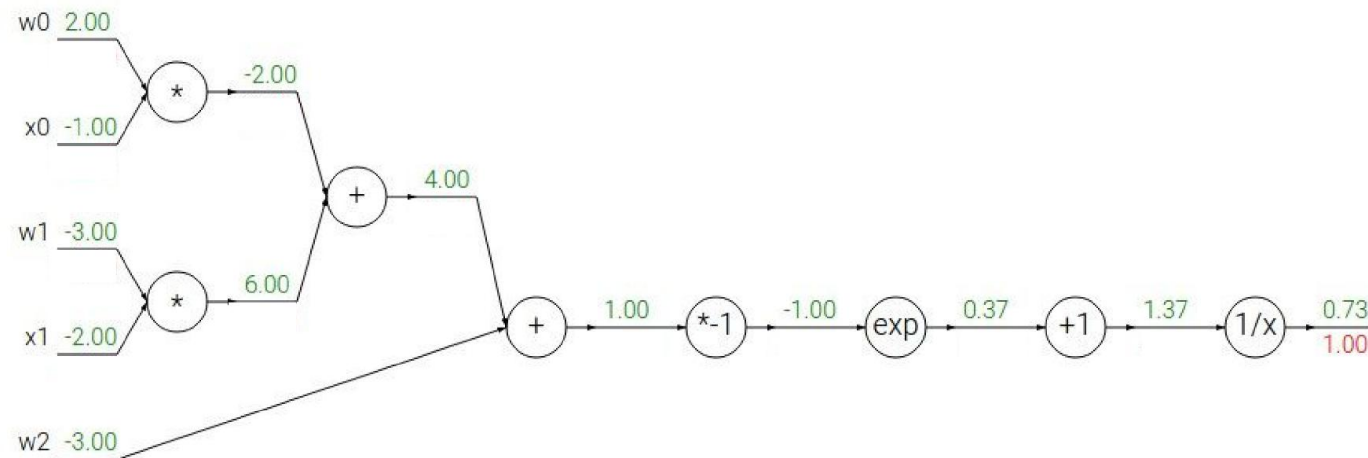




Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$

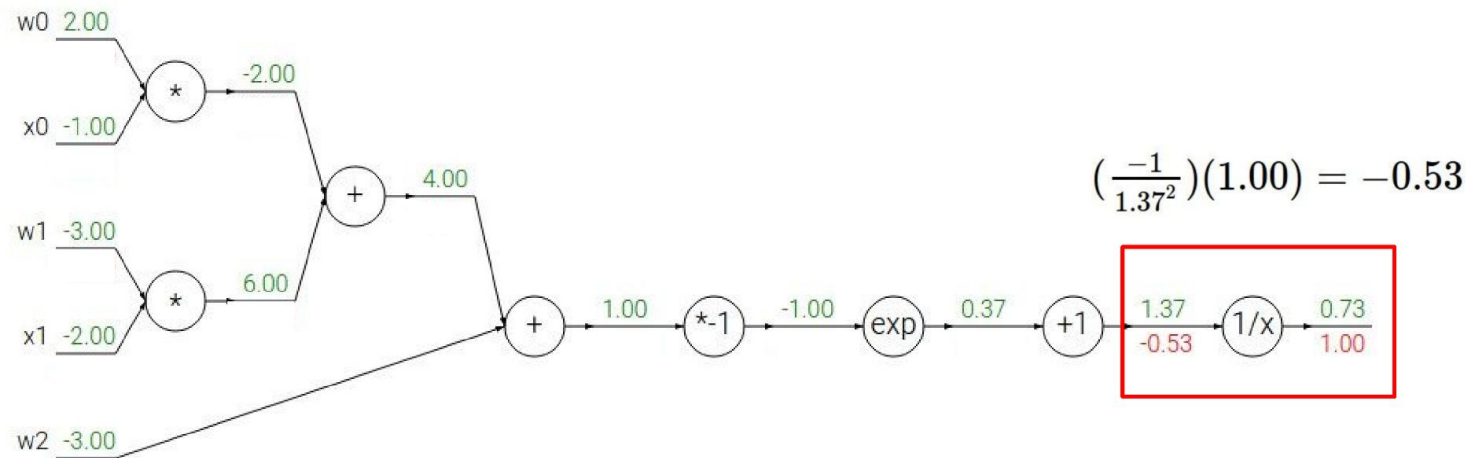


Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



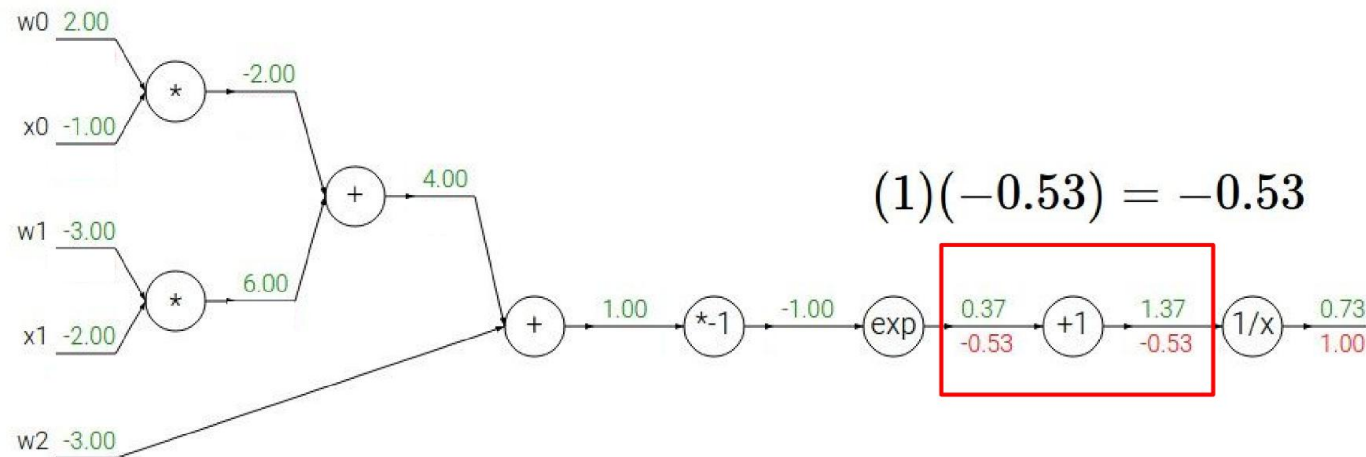
$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



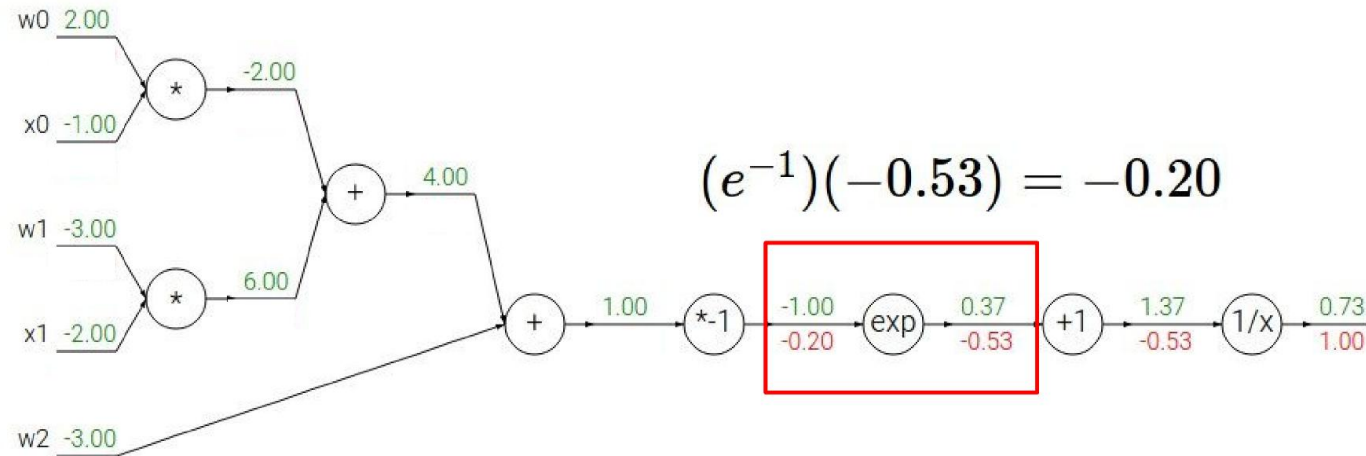
$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$f(x) = e^x$	→	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	→	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	→	$\frac{df}{dx} = a$		$f_c(x) = c + x$	→	$\frac{df}{dx} = 1$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



$$(e^{-1})(-0.53) = -0.20$$

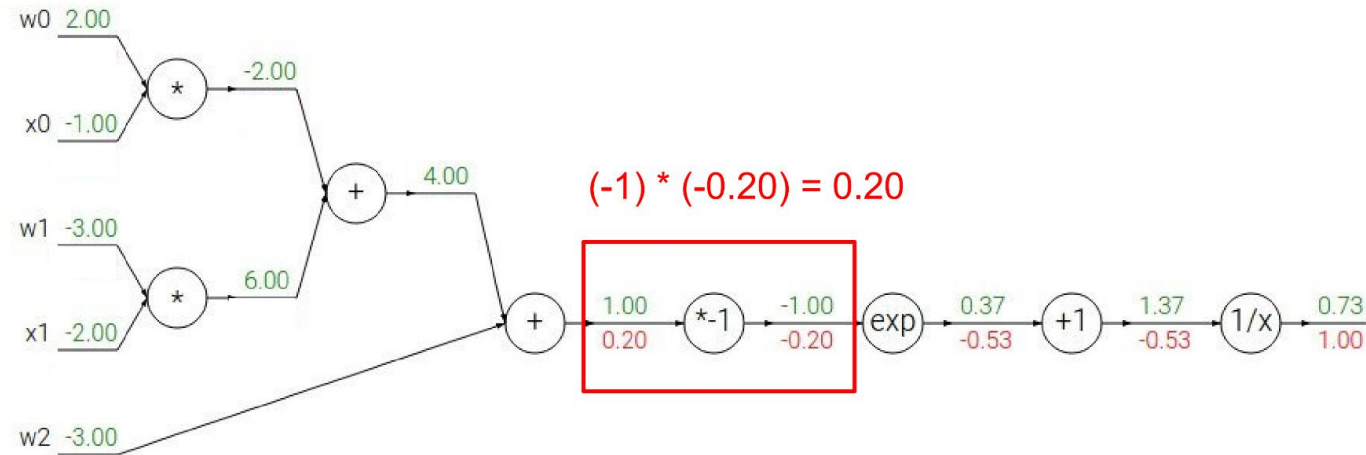
$$\boxed{f(x) = e^x \rightarrow \frac{df}{dx} = e^x}$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

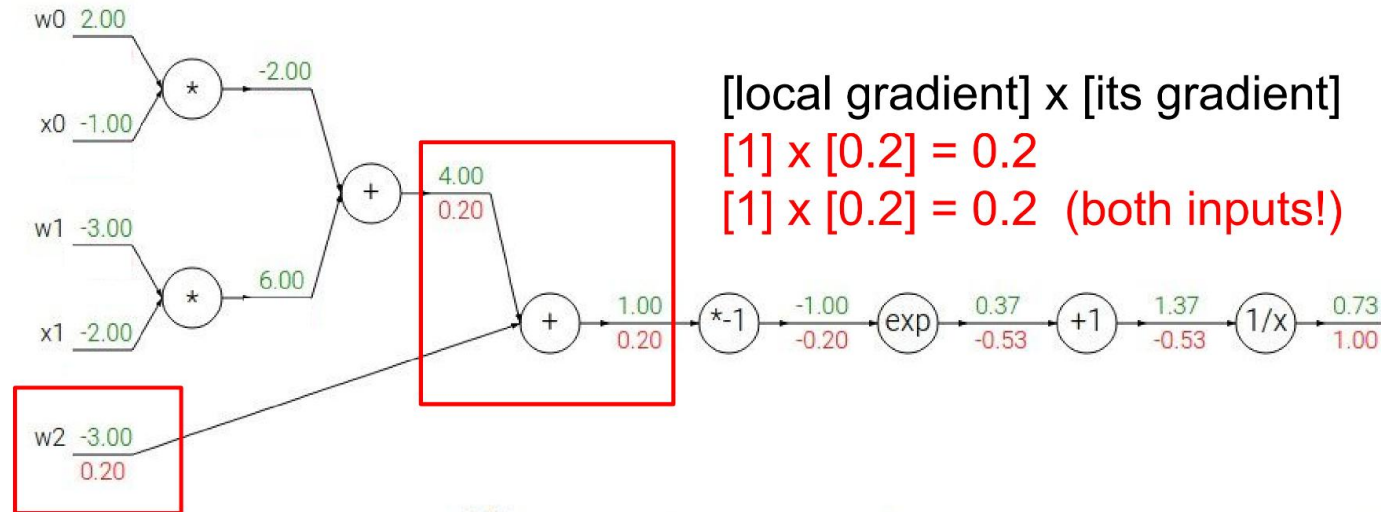
$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



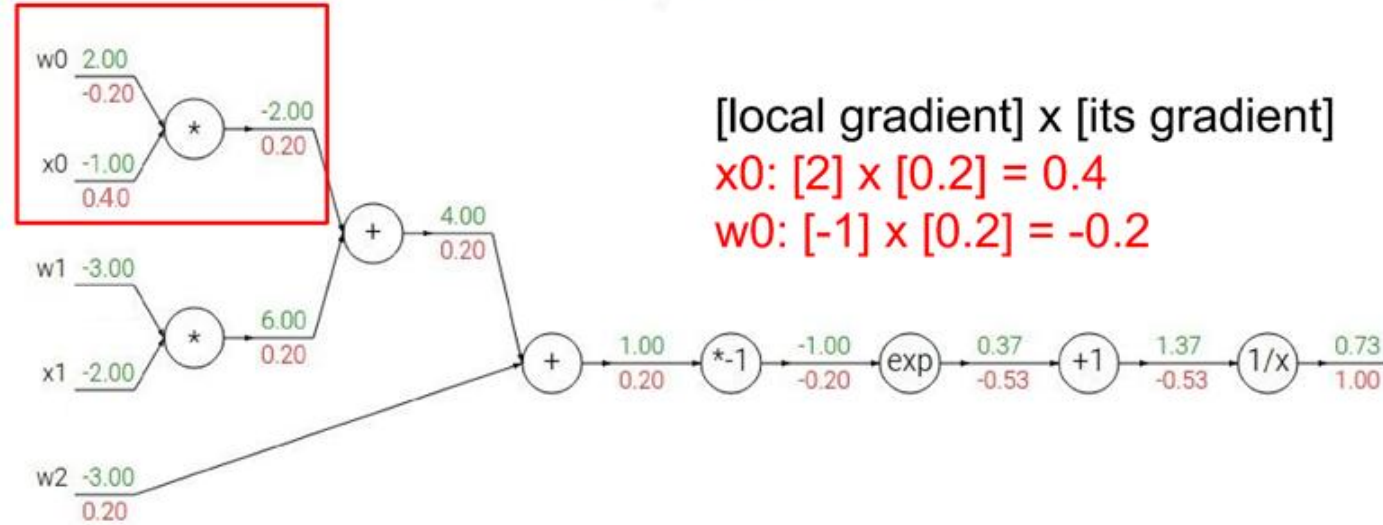
$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$

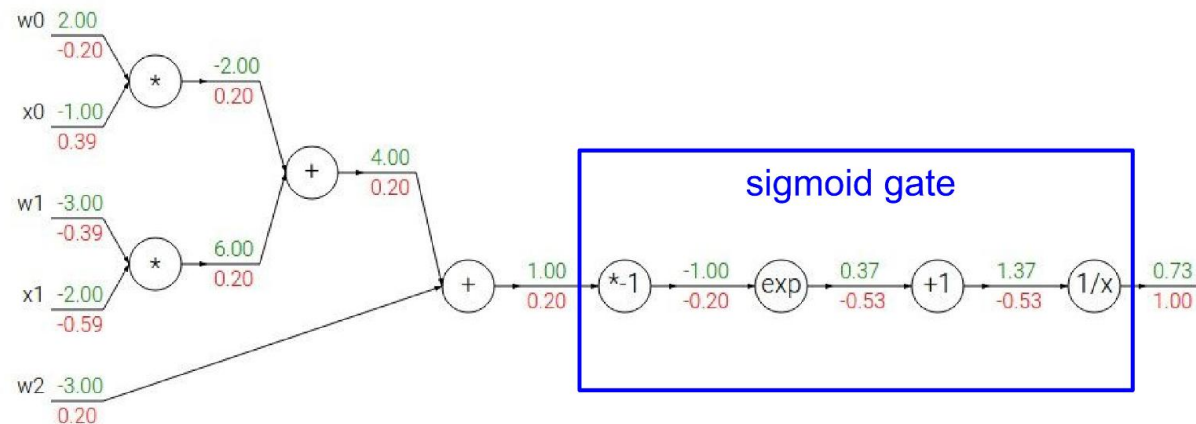


$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \text{sigmoid function}$$

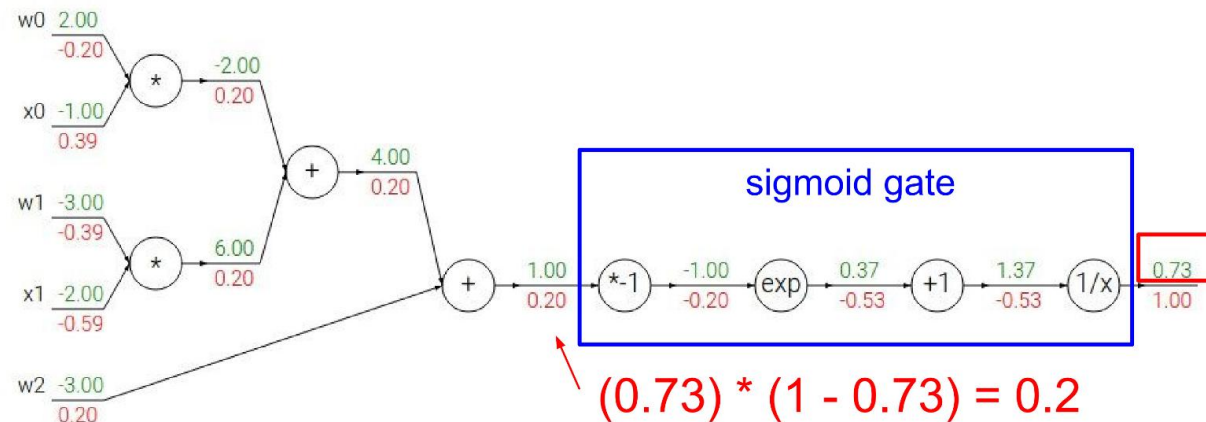
$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$



$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \text{sigmoid function}$$

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$



Revisar en Colab código de Backpropagation

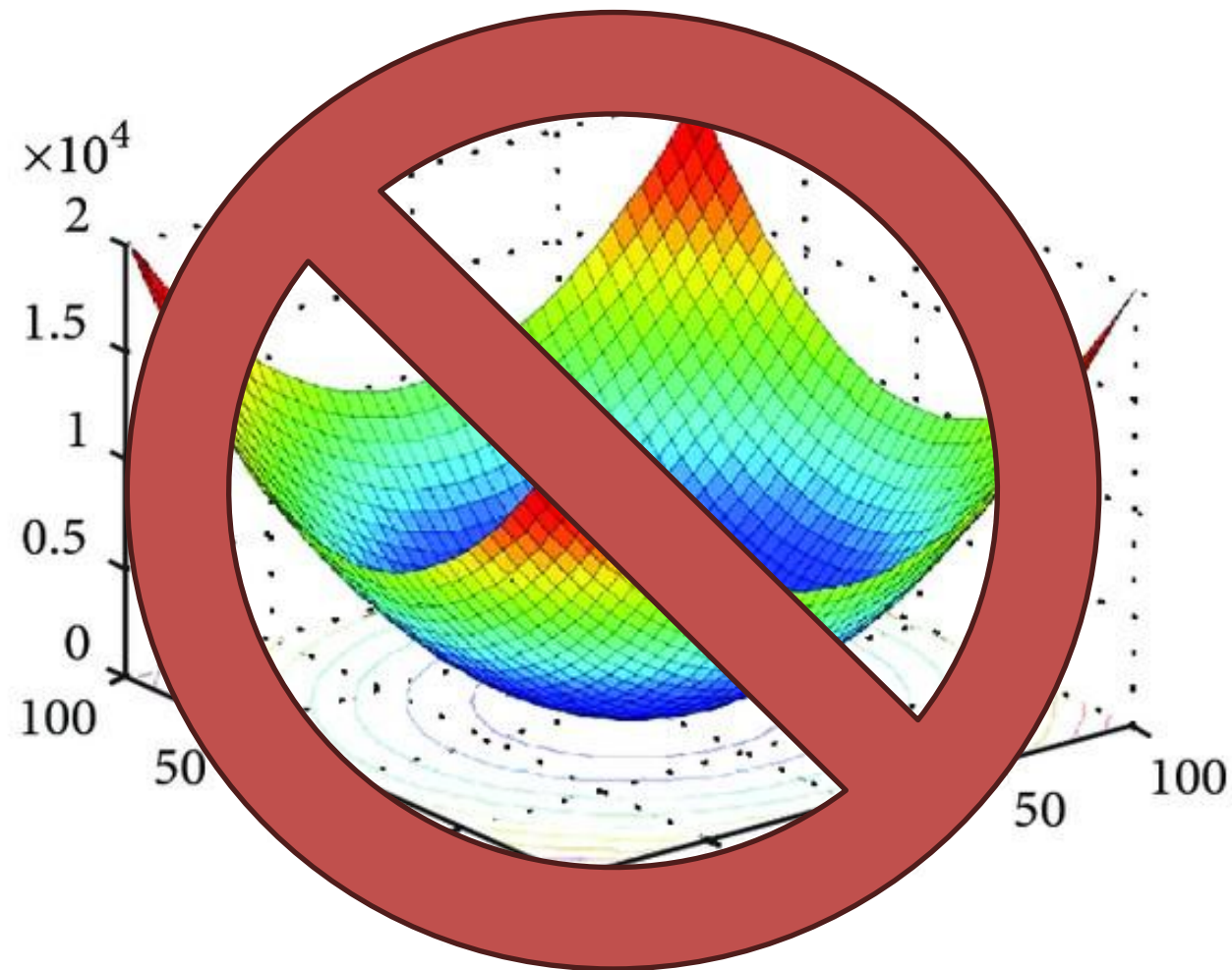


Pueden revisar también <https://github.com/karpathy/micrograd>

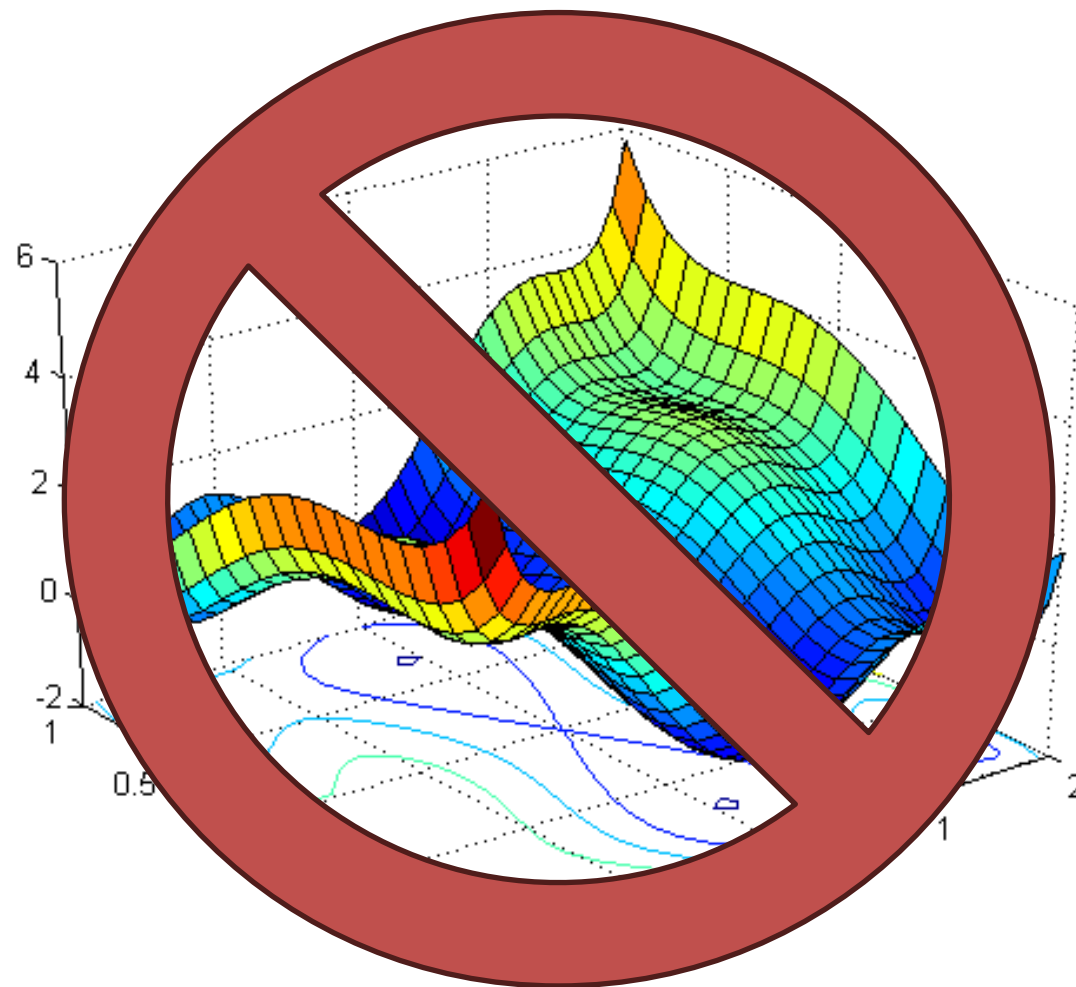
Dado el enfoque de descenso, surgen inmediatamente múltiples preguntas relevantes

Para obtener los parámetros de una red a través de un proceso de optimización, debemos considerar bastantes elementos:

- ¿Cómo calculo las derivadas para cada parámetro?
- ¿Cómo descendo de manera efectiva y eficiente en la función objetivo?
- ¿Debo inicializar los pesos y normalizar los datos de manera especial?
- ¿Puedo hacerle algo a la arquitectura de la red para facilitar el proceso?

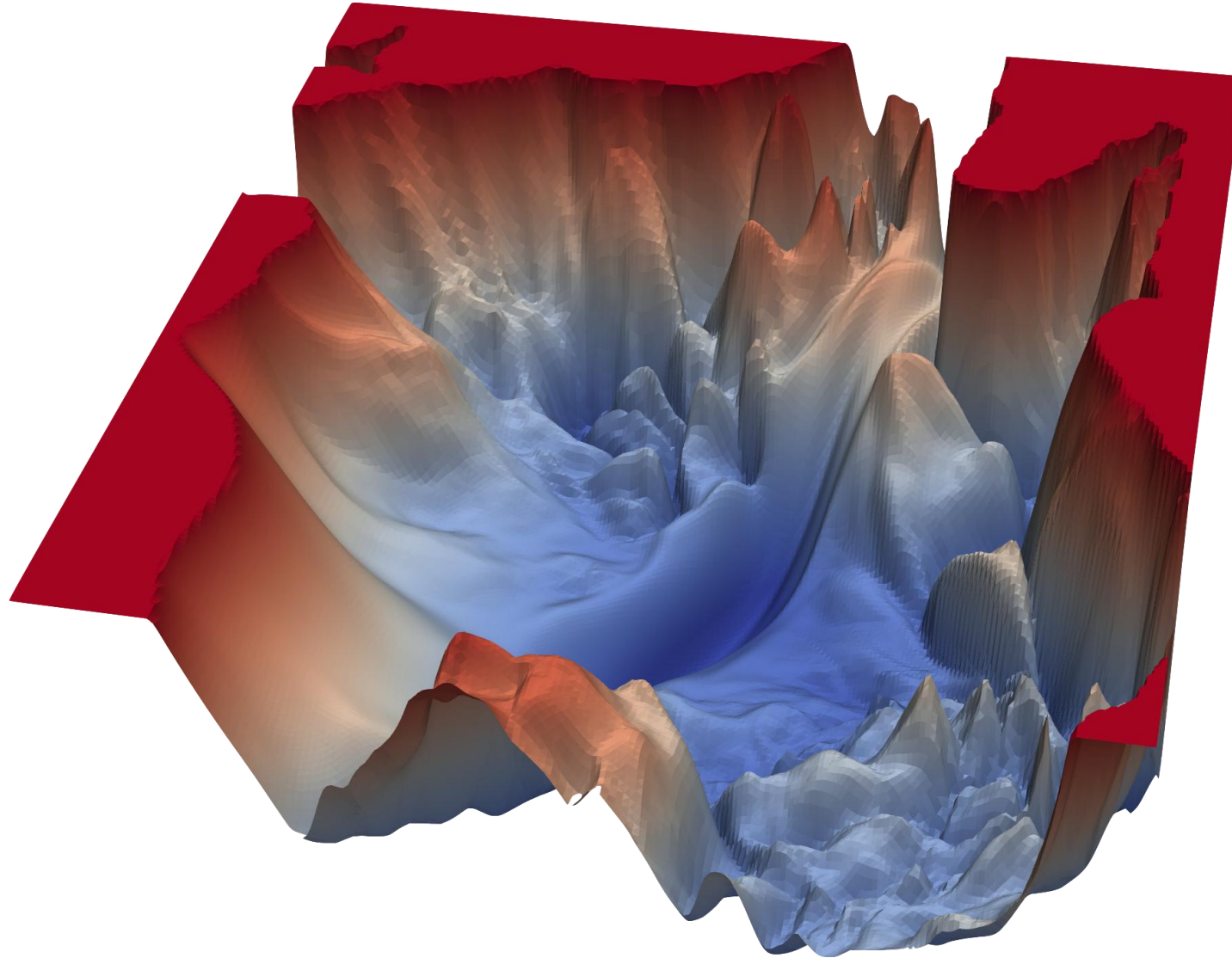


Lamentablemente, la función objetivo de una red profunda es, en general, no convexa



Además, puede contener mesetas, regiones mal condicionadas y puntos no diferenciables, dependiendo de la activación

La realidad es mucho peor (en otras palabras, el descenso de gradiente simple no funciona bien)



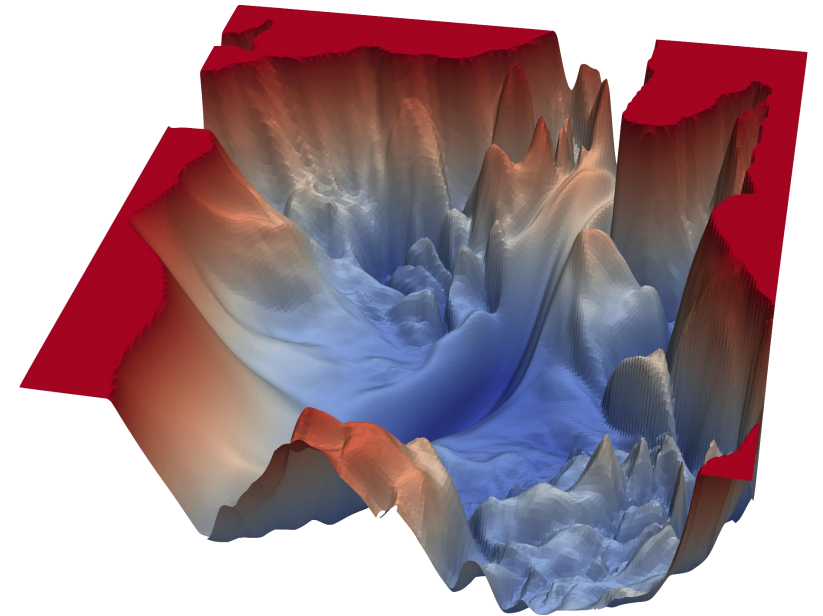


<https://youtu.be/83827jaSsGU>

<https://losslandscape.com/>

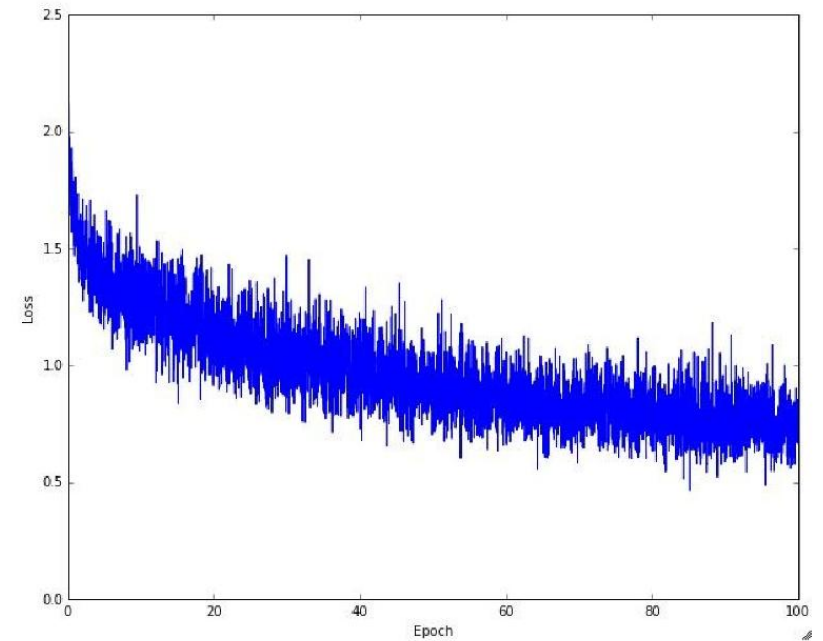
Algunos de los desafíos

- Costo de cómputo del **gradiente completo**
- Escapar de **puntos críticos** y mínimos locales
- Ajuste **dinámico** del *learning rate*
- **Escala** del learning rate para cada parámetro



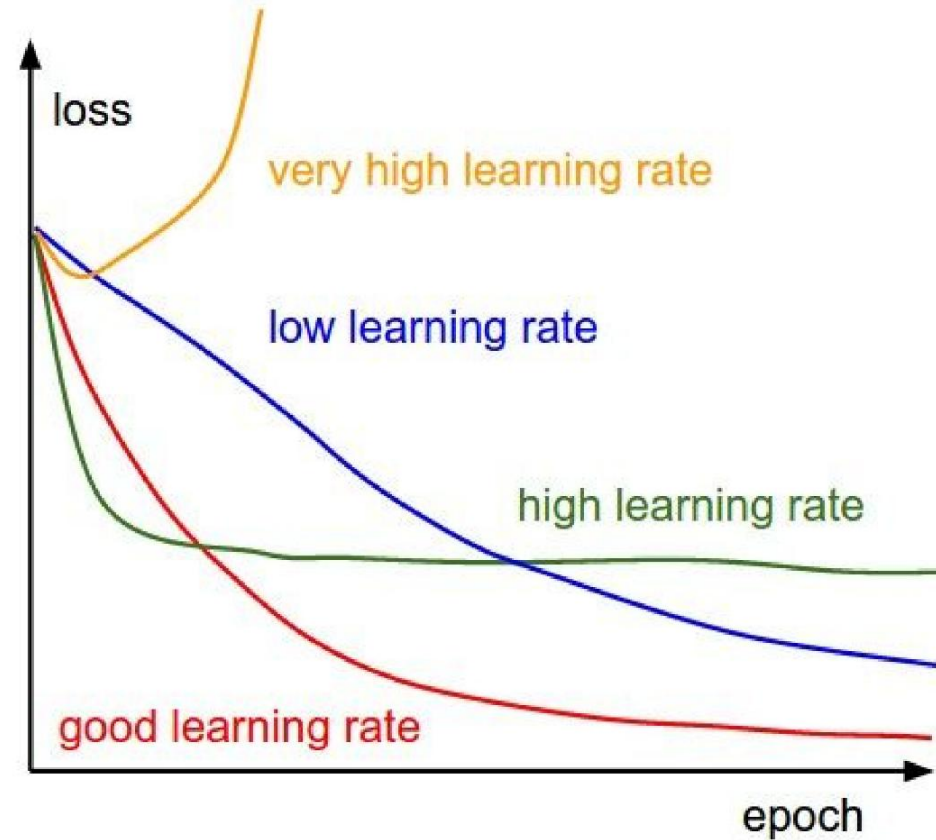
Stochastic Gradient Descent (SGD) permite entrenar modelos usando gran volumen de datos

- Datos de entrenamiento se dividen en subconjuntos disjuntos de ejemplos (*batches*).
- Usa *un batch*, no todos los ejemplos, para calcular el gradiente para cada descenso.
- Una “época” consiste en recorrer todos los *batches* del conjunto de entrenamiento.
- Bajo ciertos supuestos, SGD puede converger hacia puntos estacionarios.
- El ruido de los batches puede ayudar a abandonar algunas regiones planas o cercanas a puntos silla.



Stochastic Gradient Descent (SGD) permite entrenar modelos usando gran volumen de datos

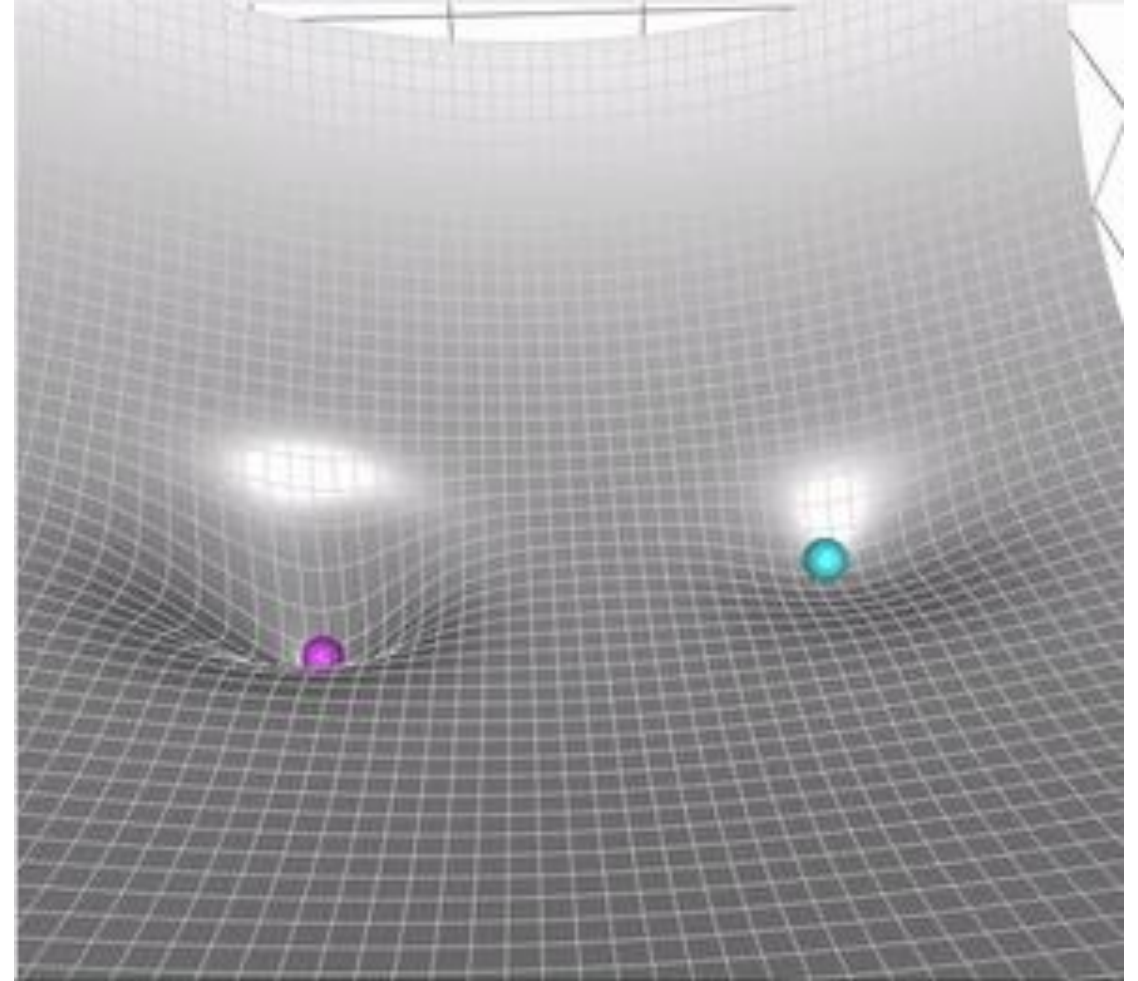
The effects of step size (or “learning rate”)



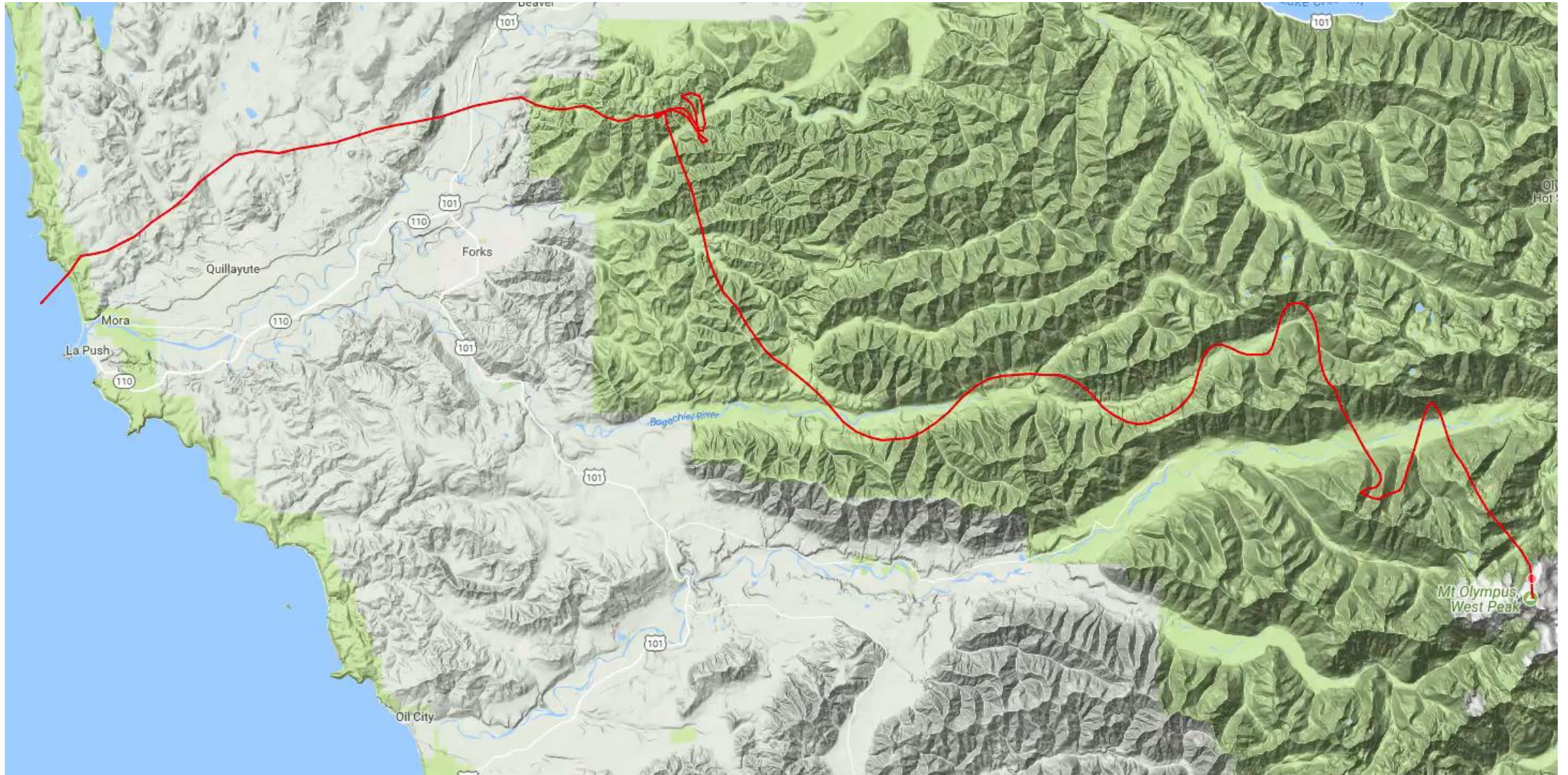
Volviendo a los puntos críticos, **momentum** permite aminorar riesgo de quedar pegado en estos

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_t$$



Un ejemplo muy “bonito” puede construirse utilizando datos geográficos de elevación

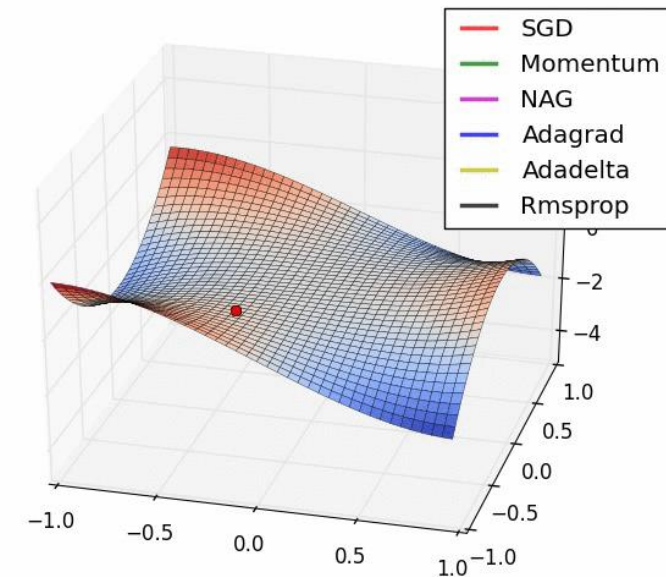
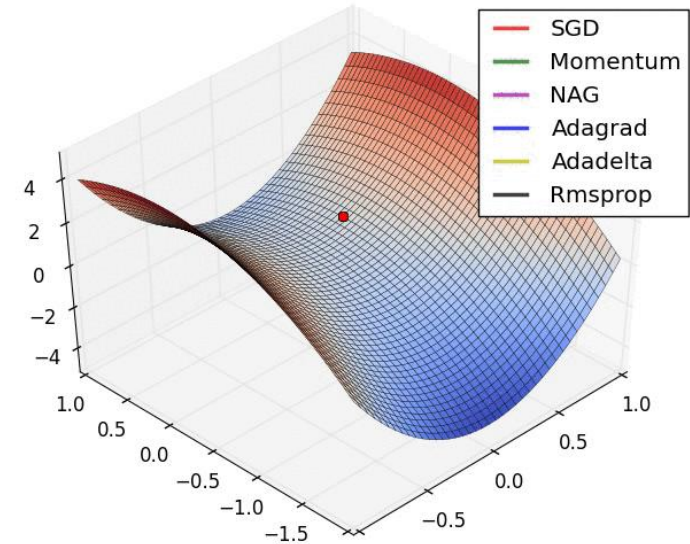


¹<https://fosterelli.co/executing-gradient-descent-on-the-earth>

Nesterov Accelerated Gradient (NAG) nos permite corregir velocidad y trayectoria del momentum, mirando hacia el futuro

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta_t - \gamma v_{t-1})$$

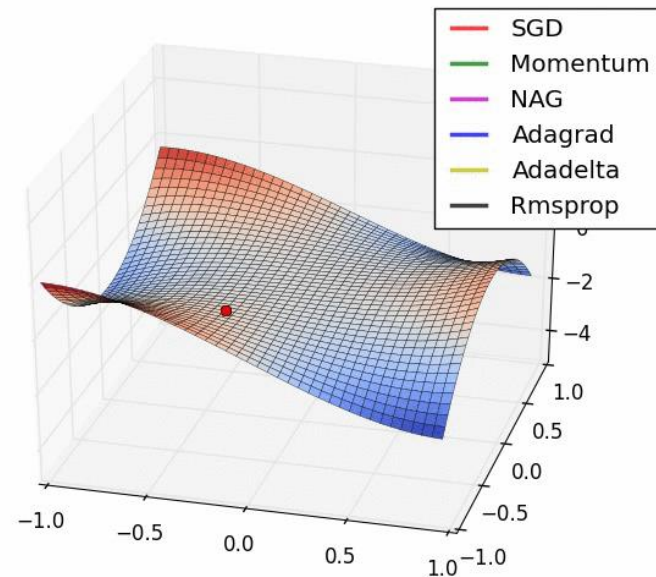
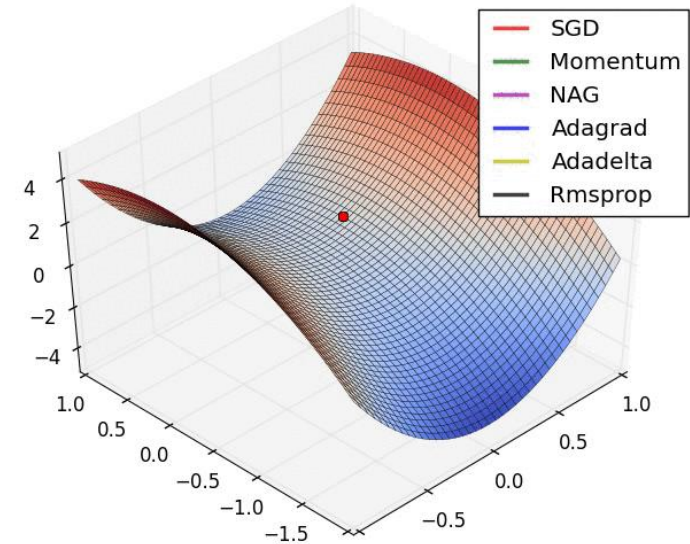
$$\theta_{t+1} = \theta_t - v_t$$



Adagrad genera *learning rates* adaptativos por parámetro, utilizando la acumulación histórica de gradientes

$$g_{t,i} = \nabla_{\theta} J(\theta_{t,i})$$

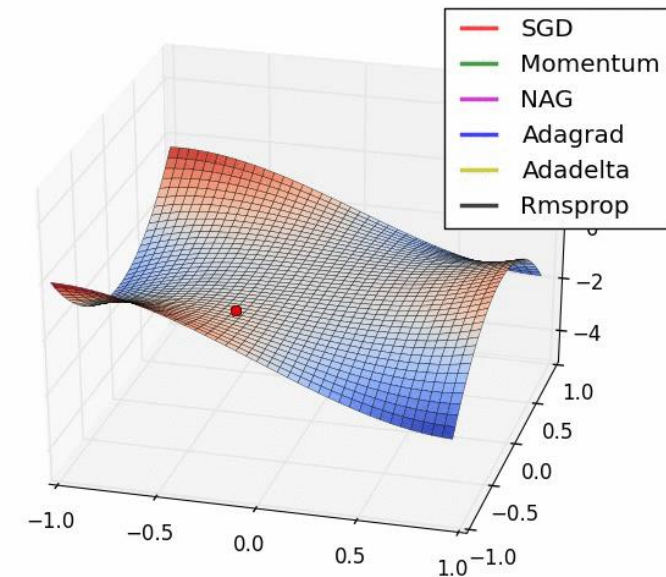
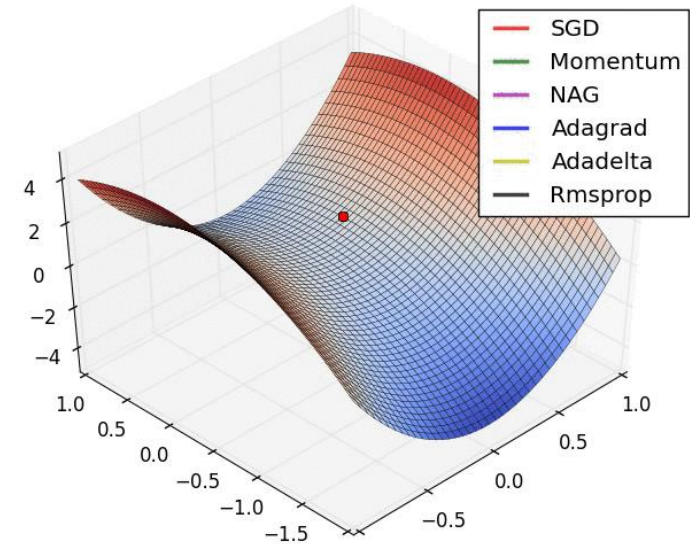
$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i} + \epsilon}} \cdot g_{t,i}$$



RMSPprop normaliza cada actualización usando un promedio móvil de gradientes cuadrados recientes

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma)g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \cdot g_t$$

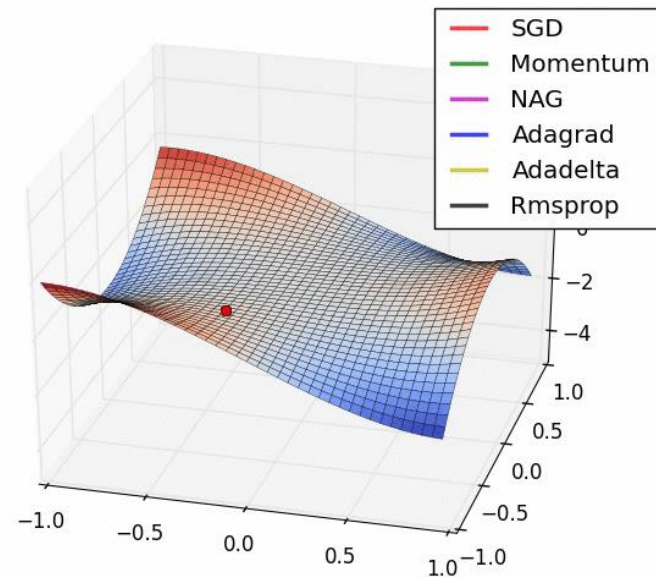
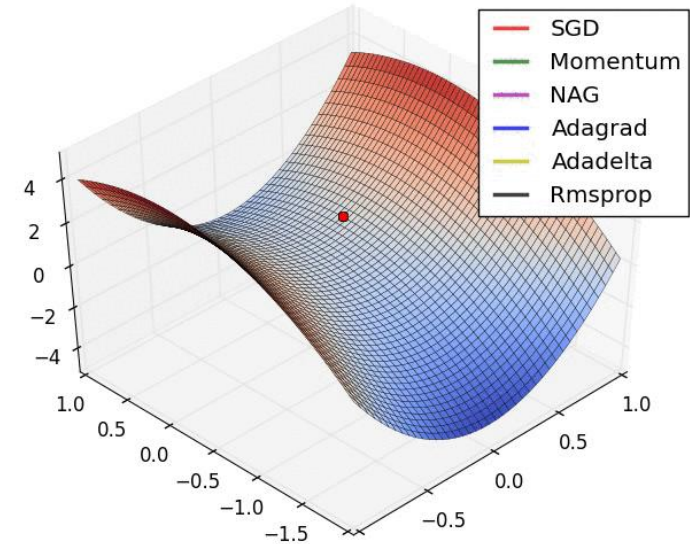


Adadelata mejora RMSprop, introduciendo un numerador para hacer calzar las “unidades”

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma)g_t^2$$

$$E[\Delta\theta^2]_t = \gamma E[\Delta\theta^2]_{t-1} + (1 - \gamma)\Delta\theta_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\sqrt{E[\Delta\theta^2]_{t-1} + \epsilon}}{\sqrt{E[g^2]_t + \epsilon}} \cdot g_t$$

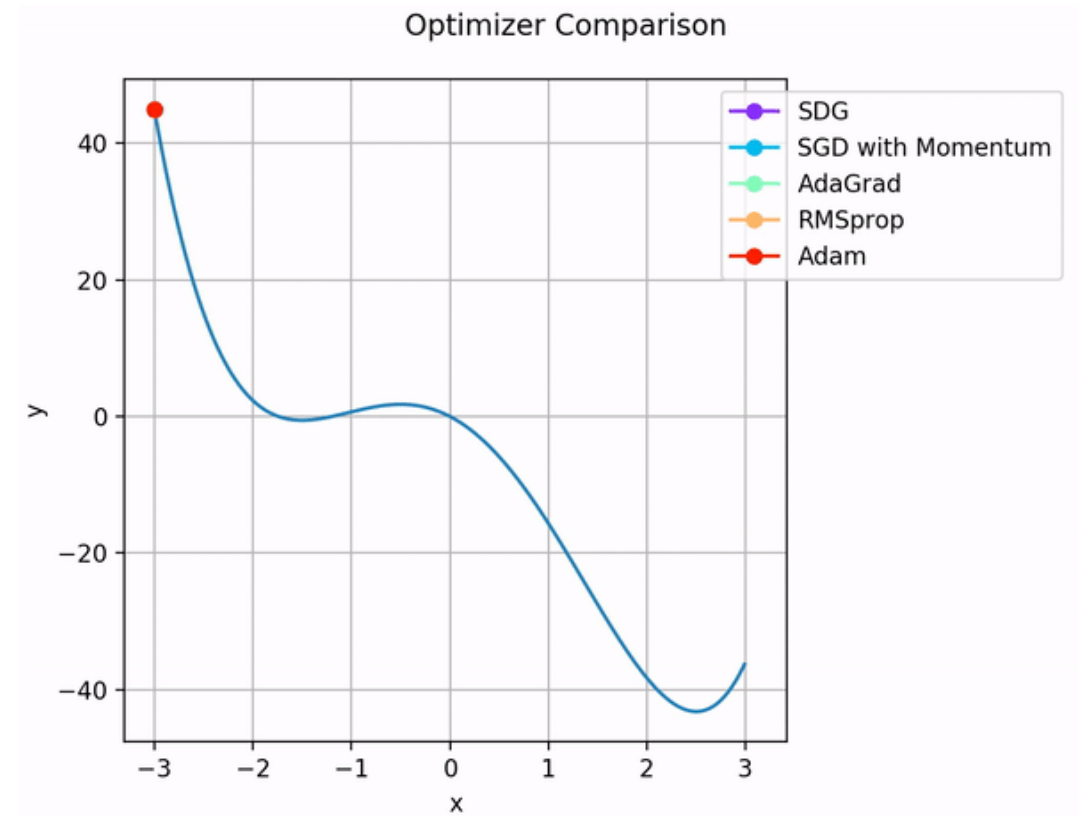


Adam combina momentum con *learning rate* adaptativo

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \rightarrow \hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \rightarrow \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$



Vamos a Colab...



Dado el enfoque de descenso, surgen inmediatamente múltiples preguntas relevantes

Para obtener los parámetros de una red a través de un proceso de optimización, debemos considerar bastantes elementos:

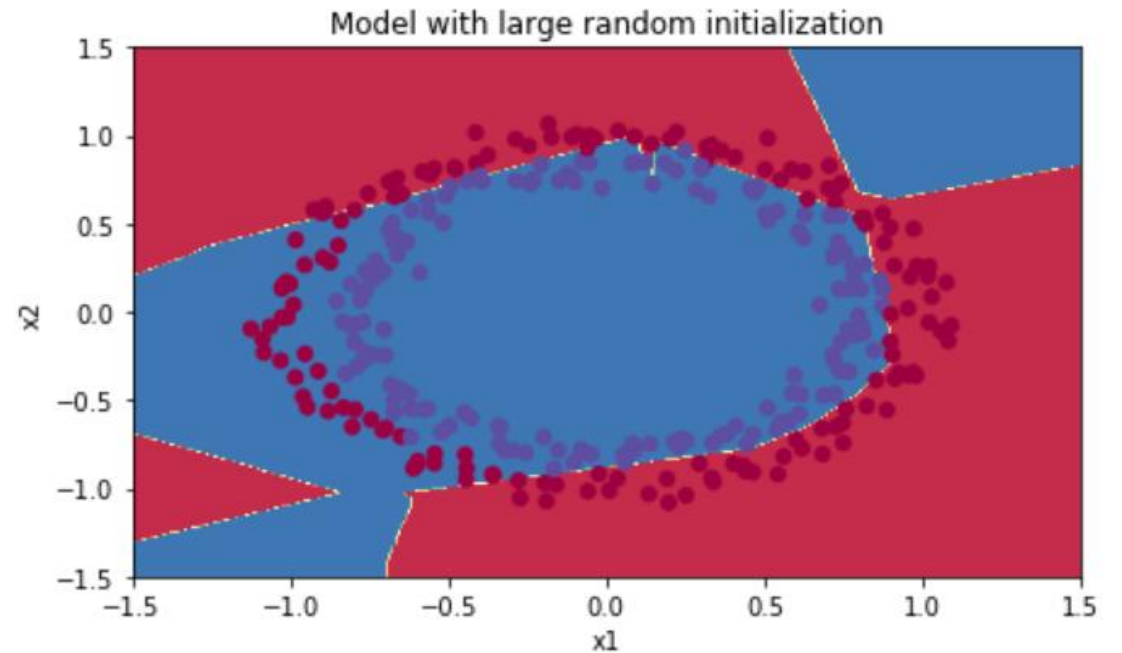
- ¿Cómo calculo las derivadas para cada parámetro?
- ¿Cómo descendo de manera efectiva y eficiente en la función objetivo?
- ¿Debo inicializar los pesos y normalizar los datos de manera especial?
- ¿Puedo hacerle algo a la arquitectura de la red para facilitar el proceso?

Inicialización de parámetros

- ¿Qué pasa si inicializamos todo con 0s?

Inicialización de parámetros

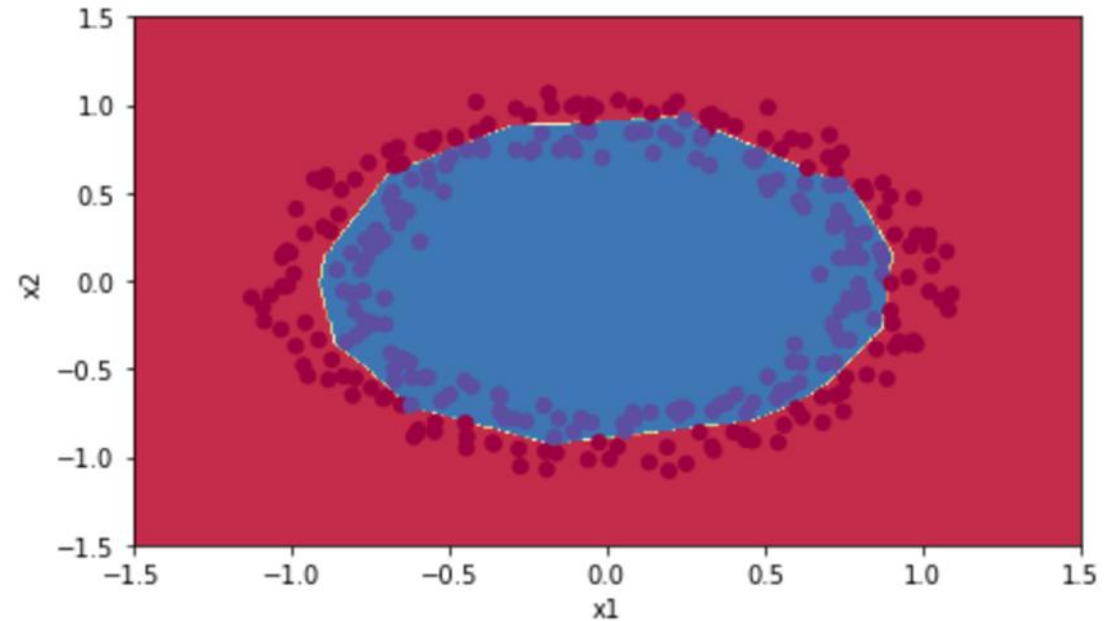
- ¿Qué pasa si inicializamos todo con 0s?
- Lo siguiente sería entonces inicializar aleatoriamente los pesos.
- El esquema aleatorio no acotado genera los siguientes problemas:
 1. Tiempo de convergencia
 2. Desvanecimiento/explosión del gradiente



Inicialización de parámetros

Para reducir estos problemas, se corrige la inicialización aleatoria, considerando el tamaño de las capas:

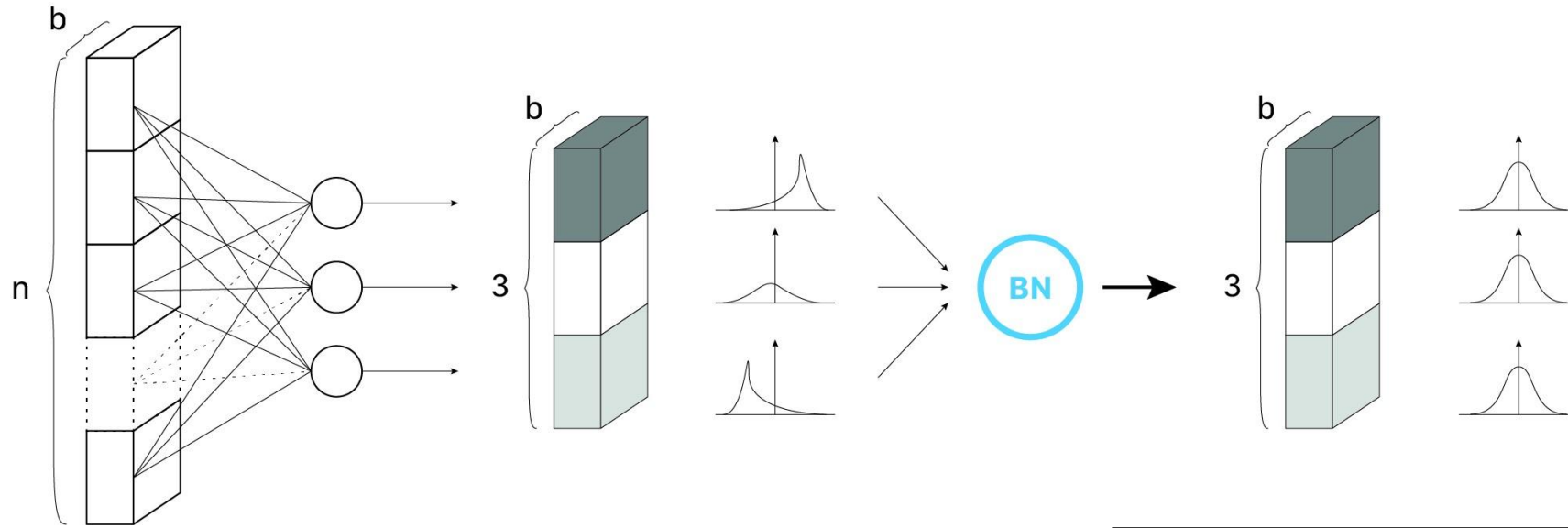
- Kaiming/Xe: $\sqrt{\frac{2}{fan_{in}}}$, usada con ReLu
- LeCun: $\sqrt{\frac{1}{fan_{in}}}$
- Xavier/Glorot: $\sqrt{\frac{2}{fan_{in}+fan_{out}}}$, usada con tanh



Capas de normalización

- La escala y variabilidad de las activaciones influyen en el condicionamiento del problema y en la estabilidad de la optimización.
- En base a esto, **normalizar** las entradas a la red es una **buena práctica**.
- Pero **¿cómo inducimos esto dentro de la red?**

Partamos por lo más simple, normalizar por *batch*



Una vez normalizadas las activaciones, las escalamos y trasladamos (parámetros aprendidos) para adecuarlas a la siguiente capa.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Capas de normalización

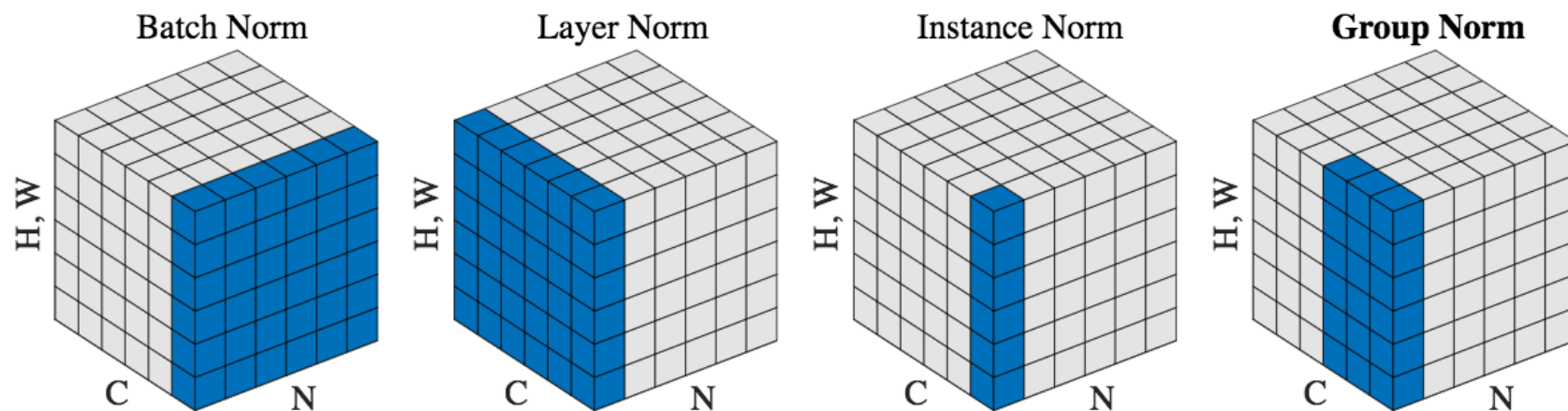


Figure 2. **Normalization methods.** Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

Vamos a Colab...

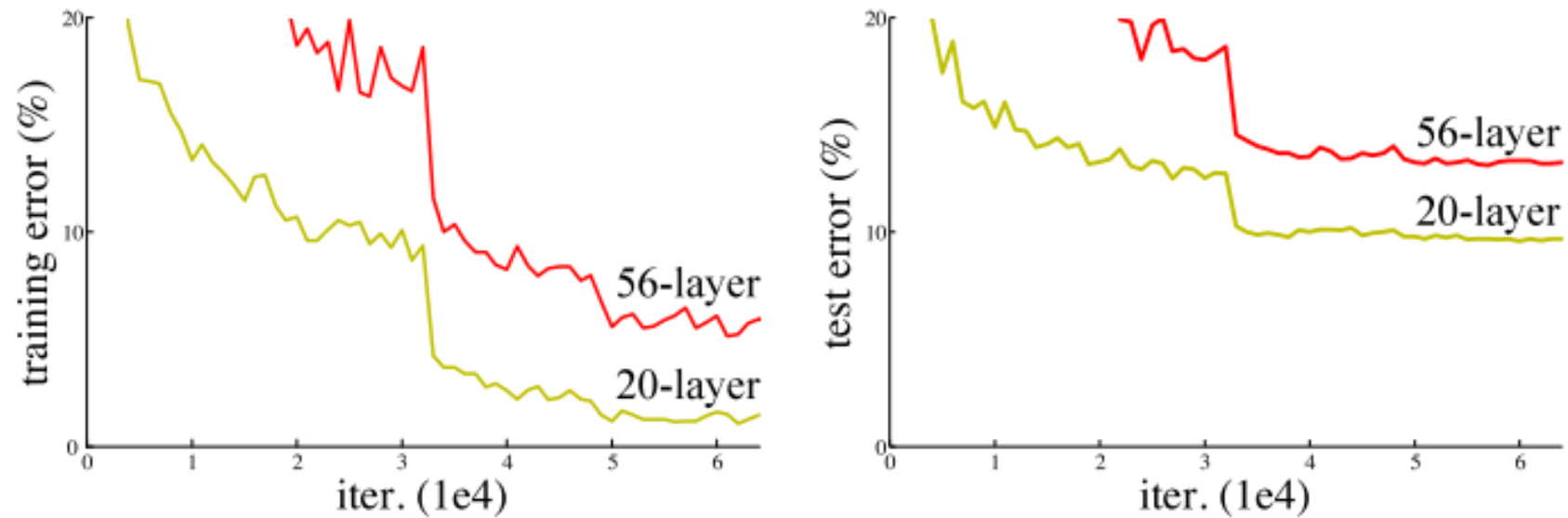


Dado el enfoque de descenso, surgen inmediatamente múltiples preguntas relevantes

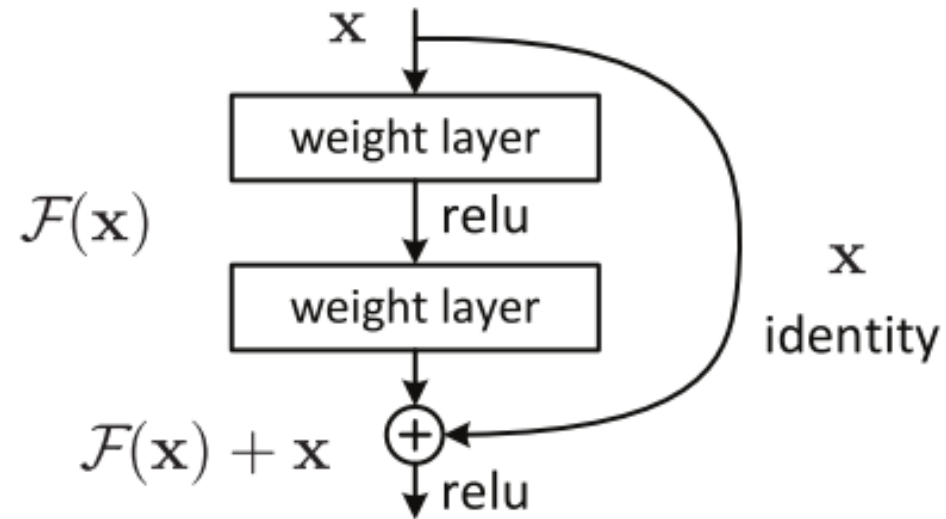
Para obtener los parámetros de una red a través de un proceso de optimización, debemos considerar bastantes elementos:

- ¿Cómo calculo las derivadas para cada parámetro?
- ¿Cómo descendo de manera efectiva y eficiente en la función objetivo?
- ¿Debo inicializar los pesos y normalizar los datos de manera especial?
- ¿Puedo hacerle algo a la arquitectura de la red para facilitar el proceso?

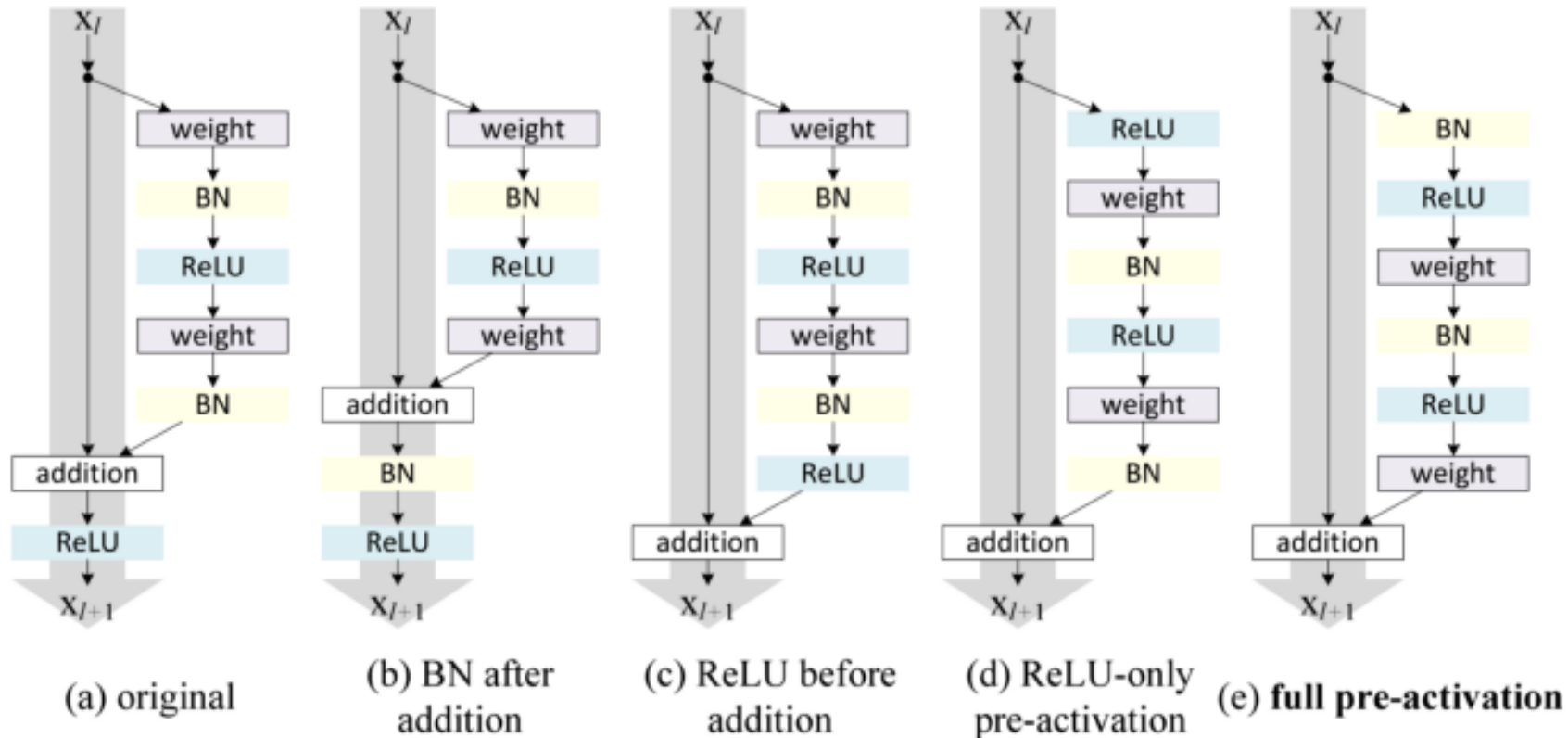
En 2015, se descubrió un fenómeno raro en las redes: más capas no significaba menor pérdida en el entrenamiento



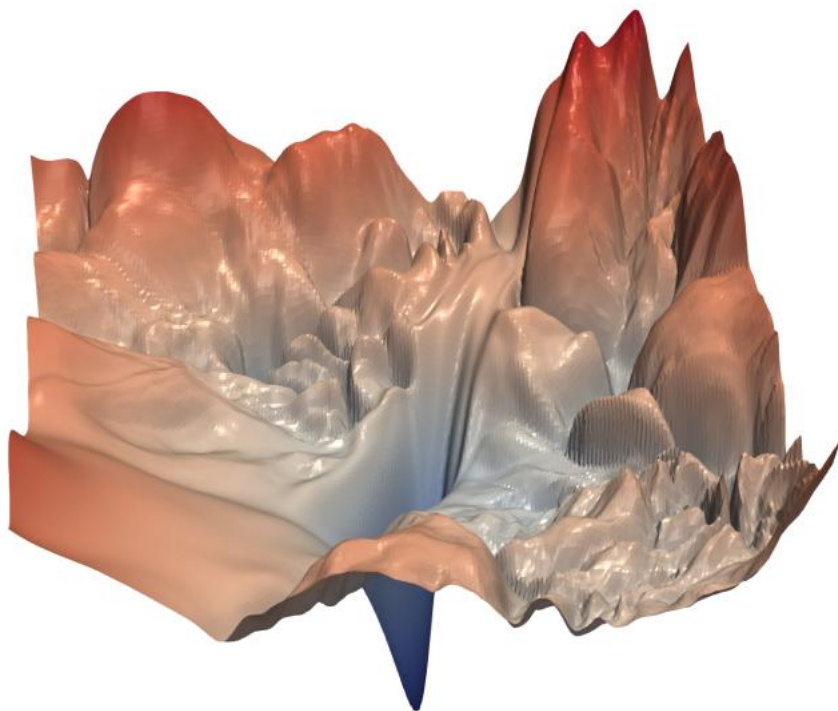
Ante la hipótesis de que el causante era la incapacidad de las redes de aprender la función identidad, se propusieron las **conexiones residuales o skip connections**



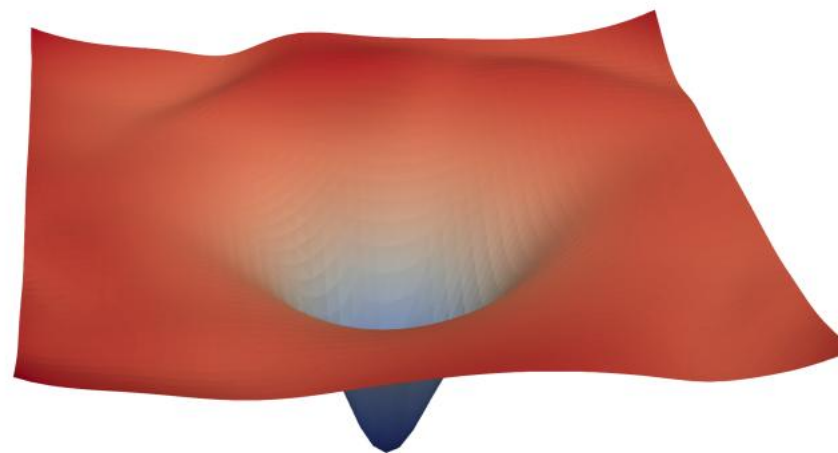
Ante la hipótesis de que el causante era la incapacidad de las redes de aprender la función identidad, se propusieron las **conexiones residuales o skip connections**



Resultados mejoran mucho gracias a un mejor condicionamiento del problema



(a) without skip connections



(b) with skip connections

Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

Resultados mejoran mucho gracias a un mejor condicionamiento del problema



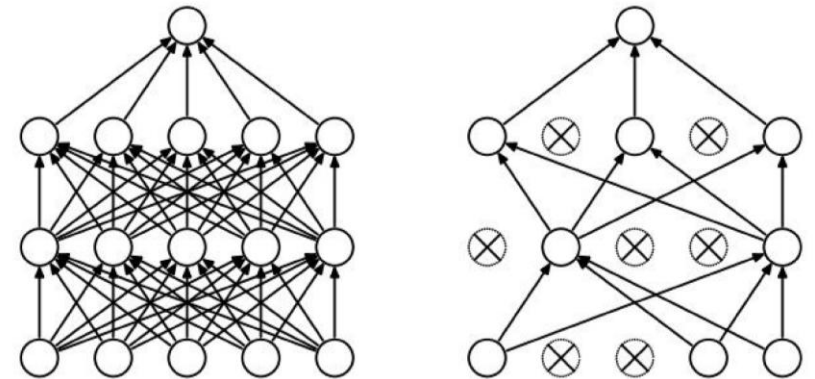
En teoría, no hay diferencia entre la teoría y la práctica. En la práctica, sí la hay.

Tras bambalinas, hacer que los modelos de DL funcionen adecuadamente, y sean prácticos, requiere una gran cantidad de detalles:

- ¿Cómo optimizar?
- ¿Cómo controlar el sobreajuste?
- ¿Cómo reutilizar?

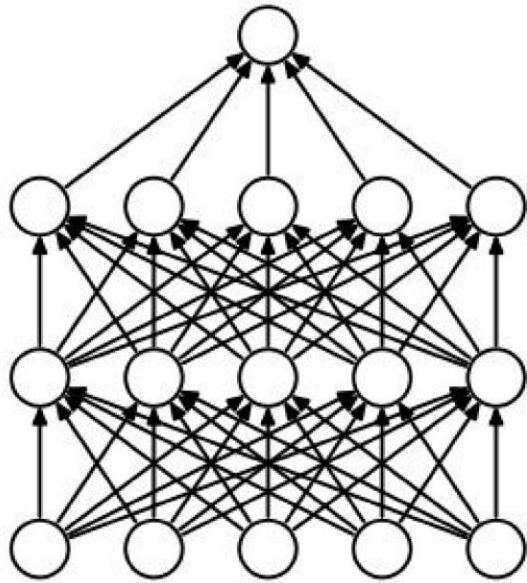
Dropout es a estar alturas un elemento muy usado en las redes

- Las capas densas del final suelen concentrar muchos parámetros y son especialmente propensas a **sobreajustarse**.
- Una estrategia habitual es aplicar **Dropout**, principalmente en estas capas.
- Durante el entrenamiento, **Dropout** apaga aleatoriamente una fracción de las activaciones.
- Esto reduce la **coadaptación** entre unidades y favorece representaciones más robustas.

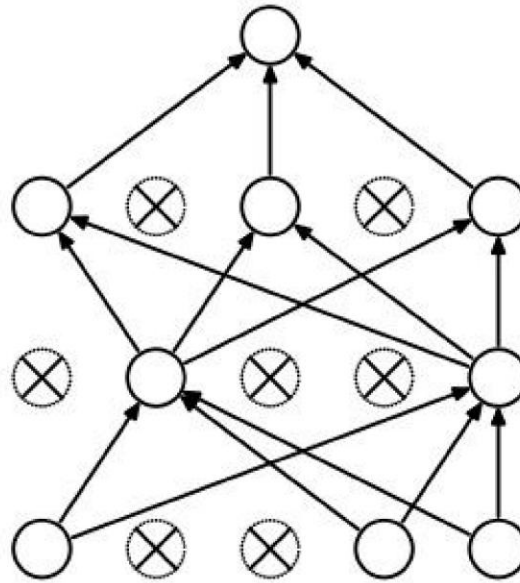


Regularization: **Dropout**

“randomly set some neurons to zero in the forward pass”



(a) Standard Neural Net

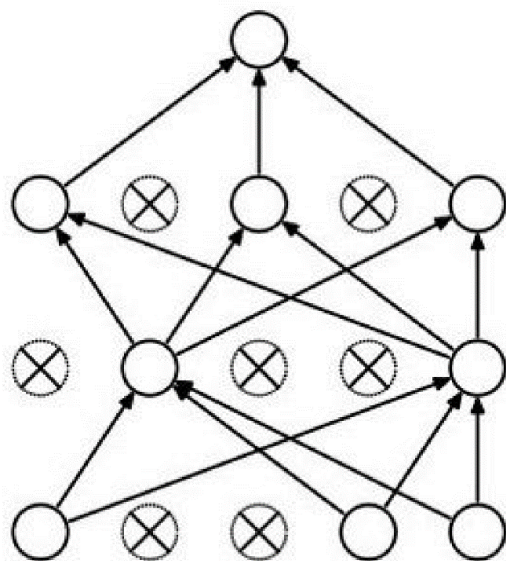


(b) After applying dropout.

[Srivastava et al., 2014]

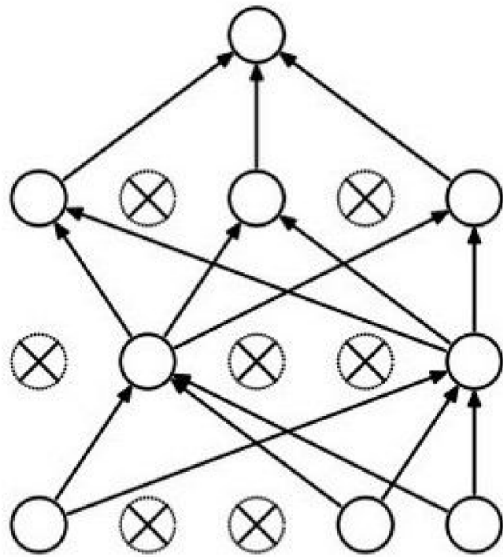
Waaaait a second...

How could this possibly be a good idea?



Waaaaait a second...

How could this possibly be a good idea?

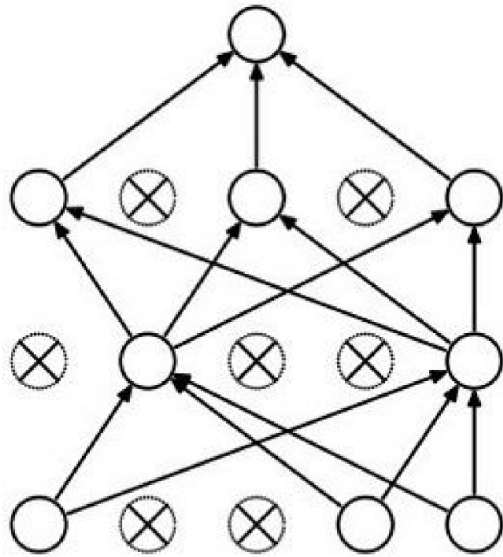


Forces the network to have a redundant representation.



Waaaaait a second...

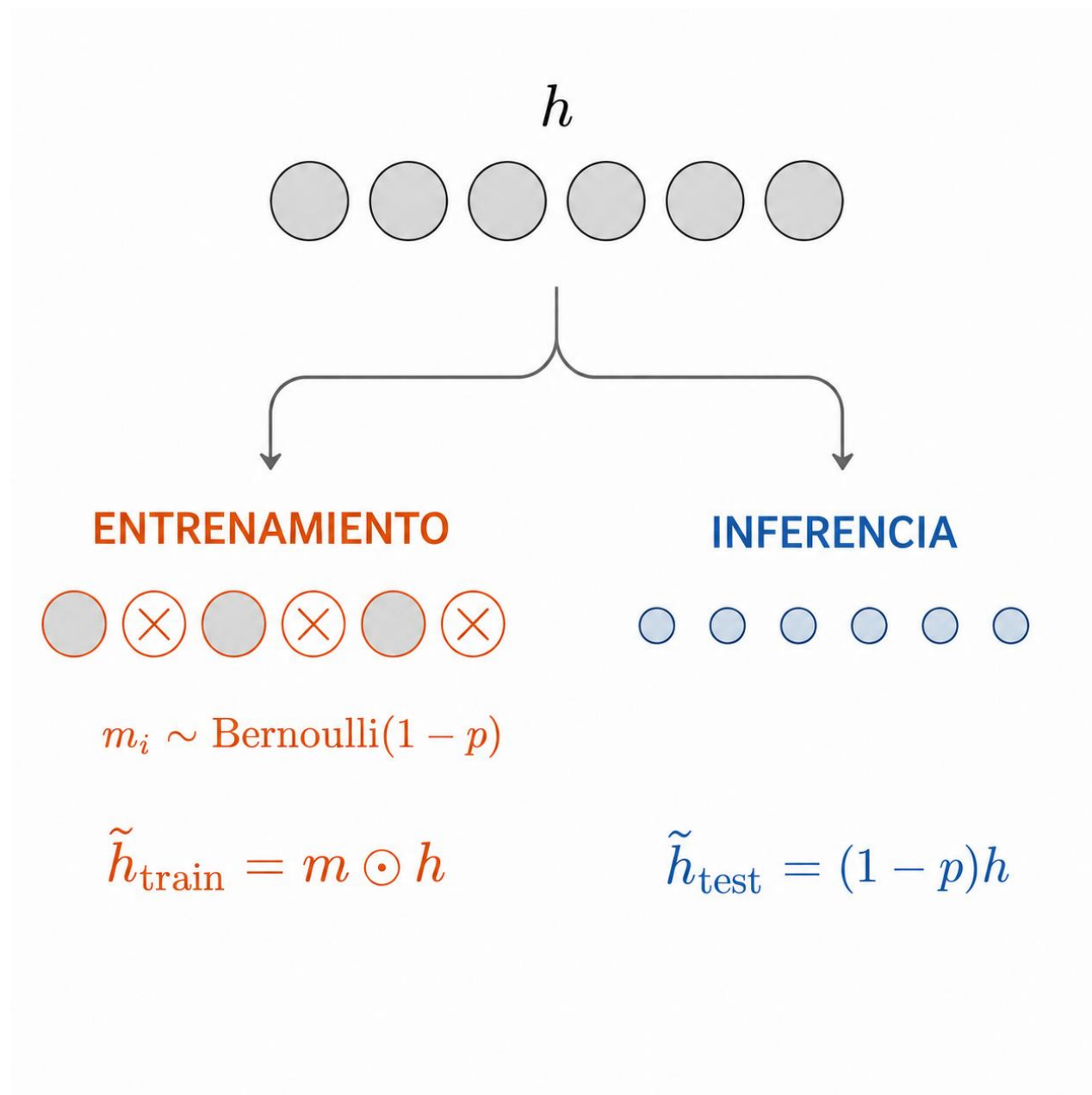
How could this possibly be a good idea?



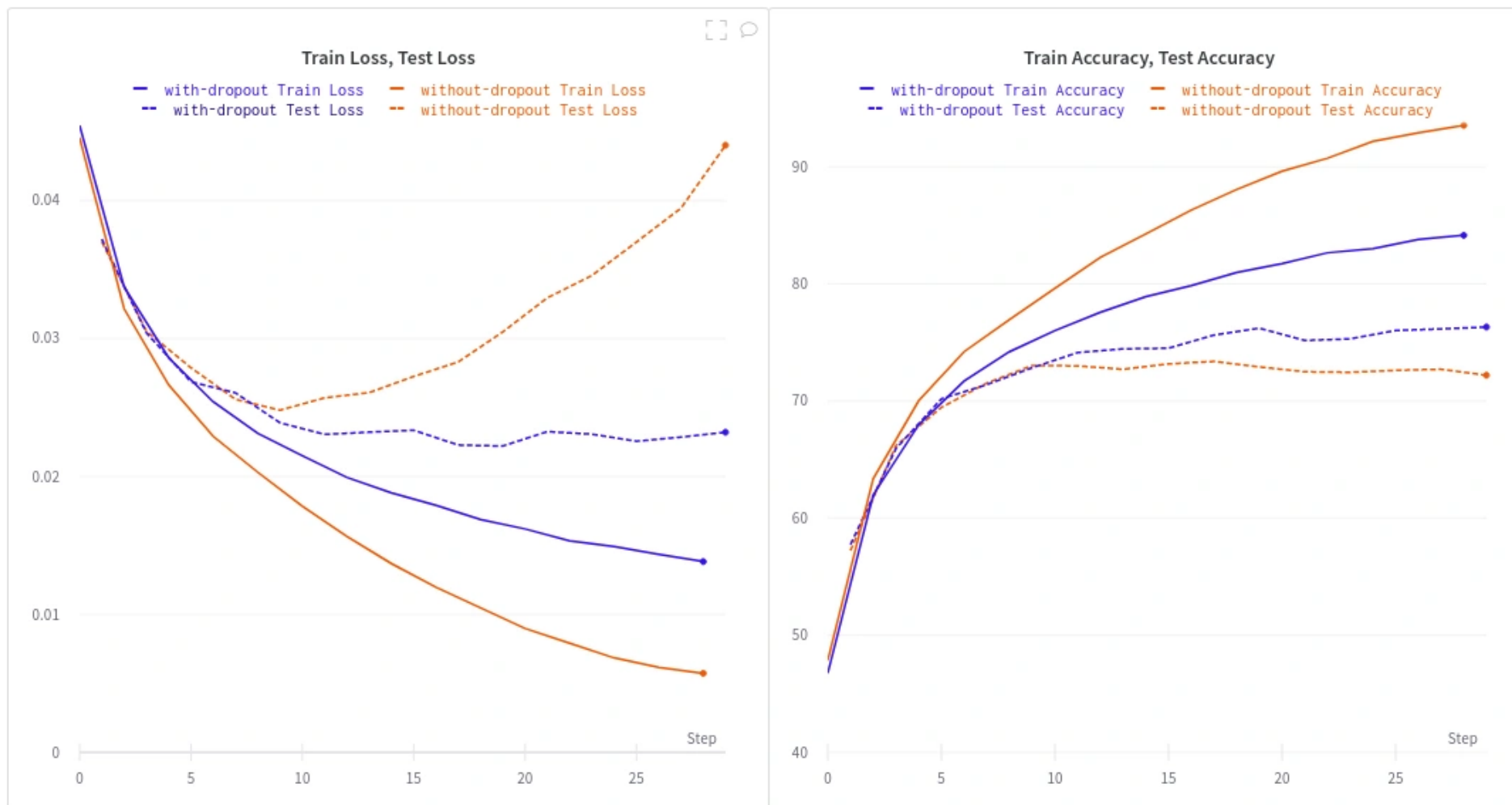
Another interpretation:

Dropout is training a large ensemble of models (that share parameters).

Each binary mask is one model, gets trained on only ~one datapoint.



Dropout puede mejorar la generalización



Otra perspectiva de Dropout, ahora desde las superficies de optimización



Regularización explícita también permite controlar el sobreajuste

$$\operatorname{argmin}_W J(X, Y; W) = \lambda \mathcal{R}(W) + \sum_i^N \mathcal{L}(x_i, y_i; W)$$



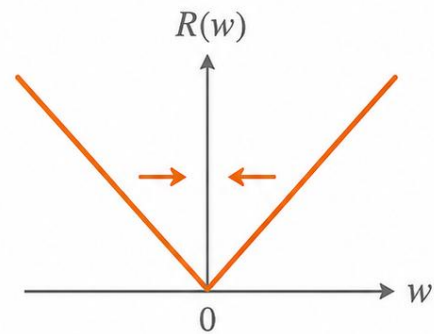
$$\ell_1 = \sum_{i,j,k} |W_{i,j,k}|$$

$$\ell_2 = \sum_{i,j,k} W_{i,j,k}^2$$

Regularización explícita también permite controlar el sobreajuste

L1

$$R(w) = \lambda |w|$$



$$\frac{\partial R}{\partial w} = \lambda \text{sign}(w)$$

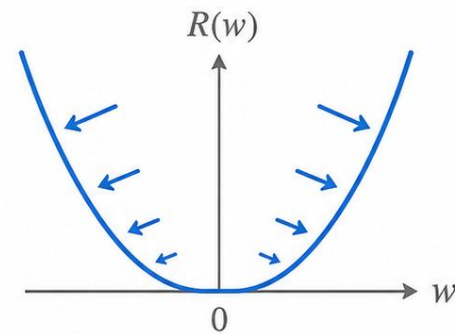
Empuje constante hacia 0



Varios pesos $\rightarrow 0$

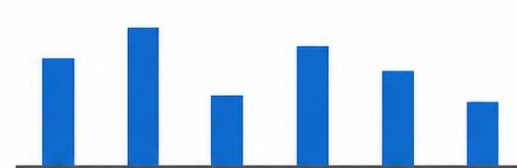
L2

$$R(w) = \lambda w^2$$



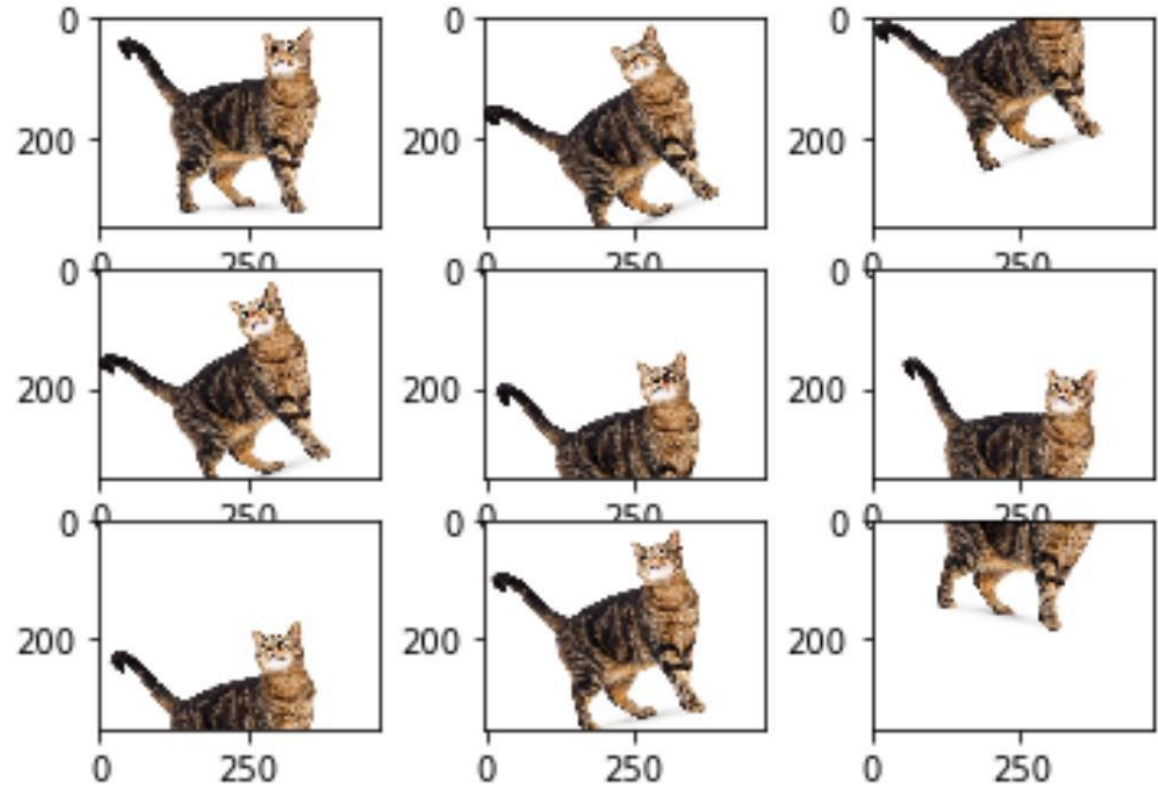
$$\frac{\partial R}{\partial w} = 2\lambda w$$

Empuje proporcional a $|w|$

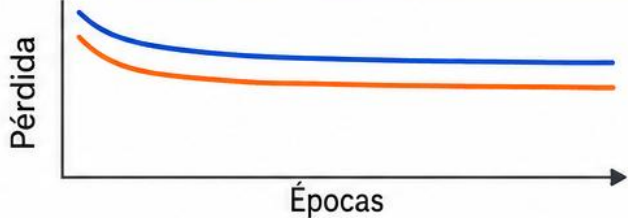
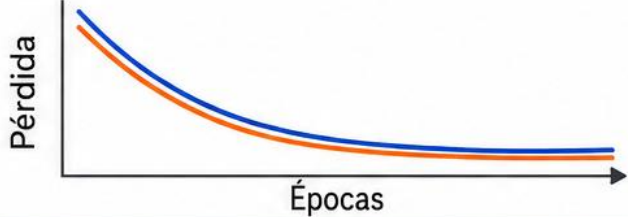
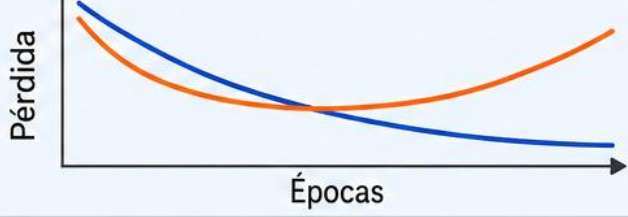
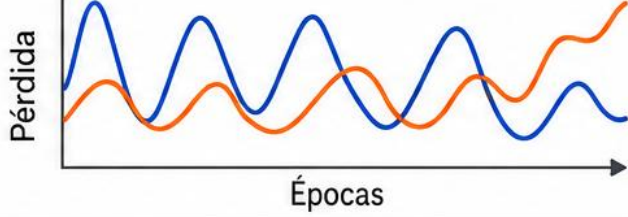


Todos los pesos se reducen

Otro de los esquemas más populares de control de sobreajuste es el aumento de datos, que incrementa el conjunto de datos “gratuitamente”



Del síntoma a la intervención

		— Entrenamiento	— Validación
SÍNTOMA	DIAGNÓSTICO	INTERVENCIÓN	
	Subajuste	 Más capacidad o entrenamiento  Menos regularización	
	Buen ajuste	 Conservar el modelo	
	Sobreajuste	 Early stopping · L1/L2 · Dropout  Más datos	
	Entrenamiento inestable	 Revisar tasa de aprendizaje, inicialización y normalización	

Vamos a Colab...



En teoría, no hay diferencia entre la teoría y la práctica. En la práctica, sí la hay.

Tras bambalinas, hacer que los modelos de DL funcionen adecuadamente, y sean prácticos, requiere una gran cantidad de detalles:

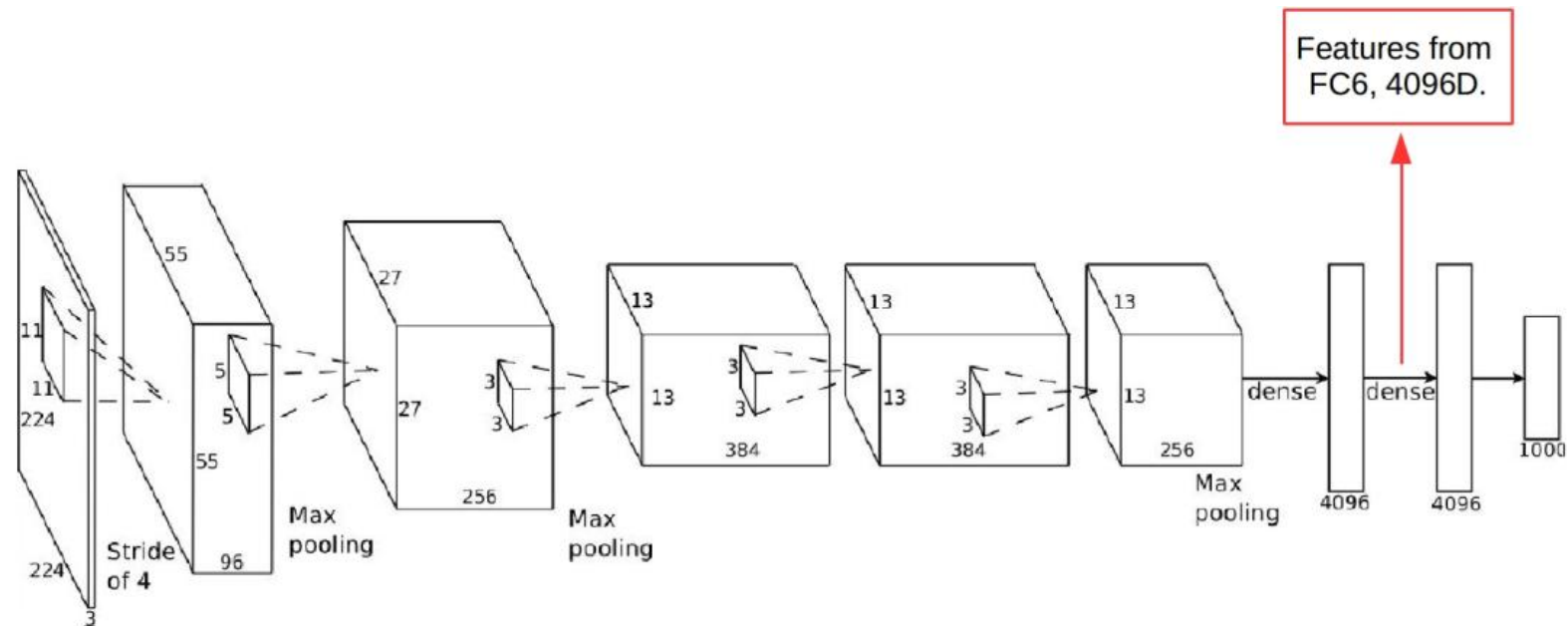
- ¿Cómo optimizar?
- ¿Cómo controlar el sobreajuste?
- ¿Cómo reutilizar?

La reutilización revela representaciones transferibles

- Resultados de DL con imágenes son claramente impresionantes, pero presentan un problema muy puntual, que no necesariamente se alinea con muchos casos de uso reales.
- Afortunadamente, las redes neuronales profundas ya entrenadas pueden ser usadas/adaptadas con poco esfuerzo en otras tareas.
- Esta propiedad es una consecuencia de la habilidad de las redes para aprender *features* jerárquicas y composicionales.

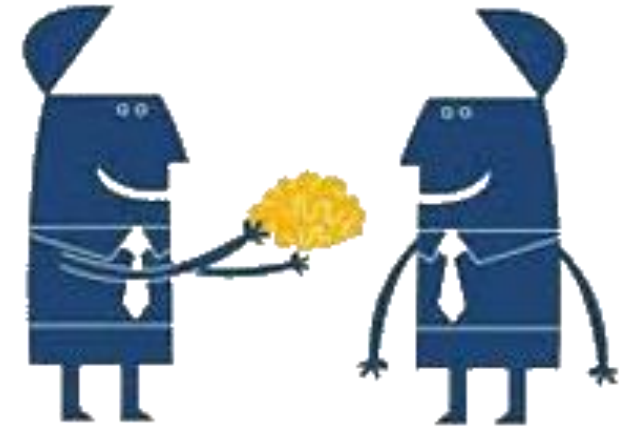
Reutilización es la máxima evidencia de generalización

- Un simple ejemplo podemos verlo con AlexNet.
- Si le “cortamos la cabeza” y la utilizamos como extractor de características, se obtienen excelentes resultados en una gran cantidad de tareas visuales: reconocimiento de escenas, caras, acciones, personas, etc.



Este tipo de ideas son parte de un área llamada *Transfer Learning*

- En resumen, la idea es aprender inicialmente modelos poderosos utilizando sets de datos abundantes y de buena calidad.
- Luego, se transfiere de alguna manera el aprendizaje a una nueva tarea, generalmente con un conjunto de datos más limitado, ya sea en cuanto a cantidad, calidad o seguridad.
- Cómo y cuándo son temas de profunda investigación y con gran relevancia en muchos problemas, como por ejemplo el desarrollo de vehículos autónomos.



Este tipo de ideas son parte de un área llamada *Transfer Learning*

Source Domain

- Lots of labeled data

$$P_S = (X_S, Y_S)$$



Target Domain

- Limited labels

$$P_T = (X_T, Y_T)$$



Hope is that:

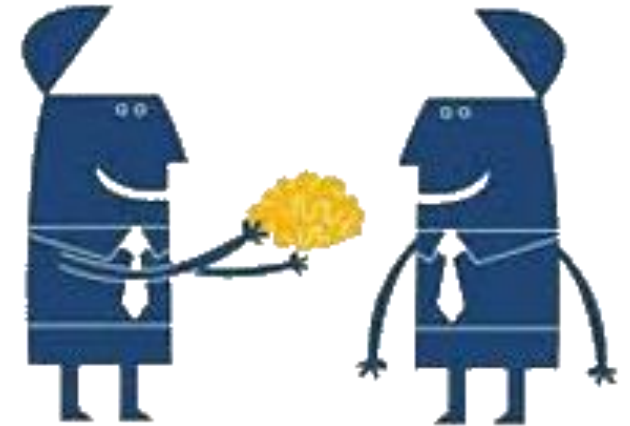
$$P_T \approx P_S$$

Or at least

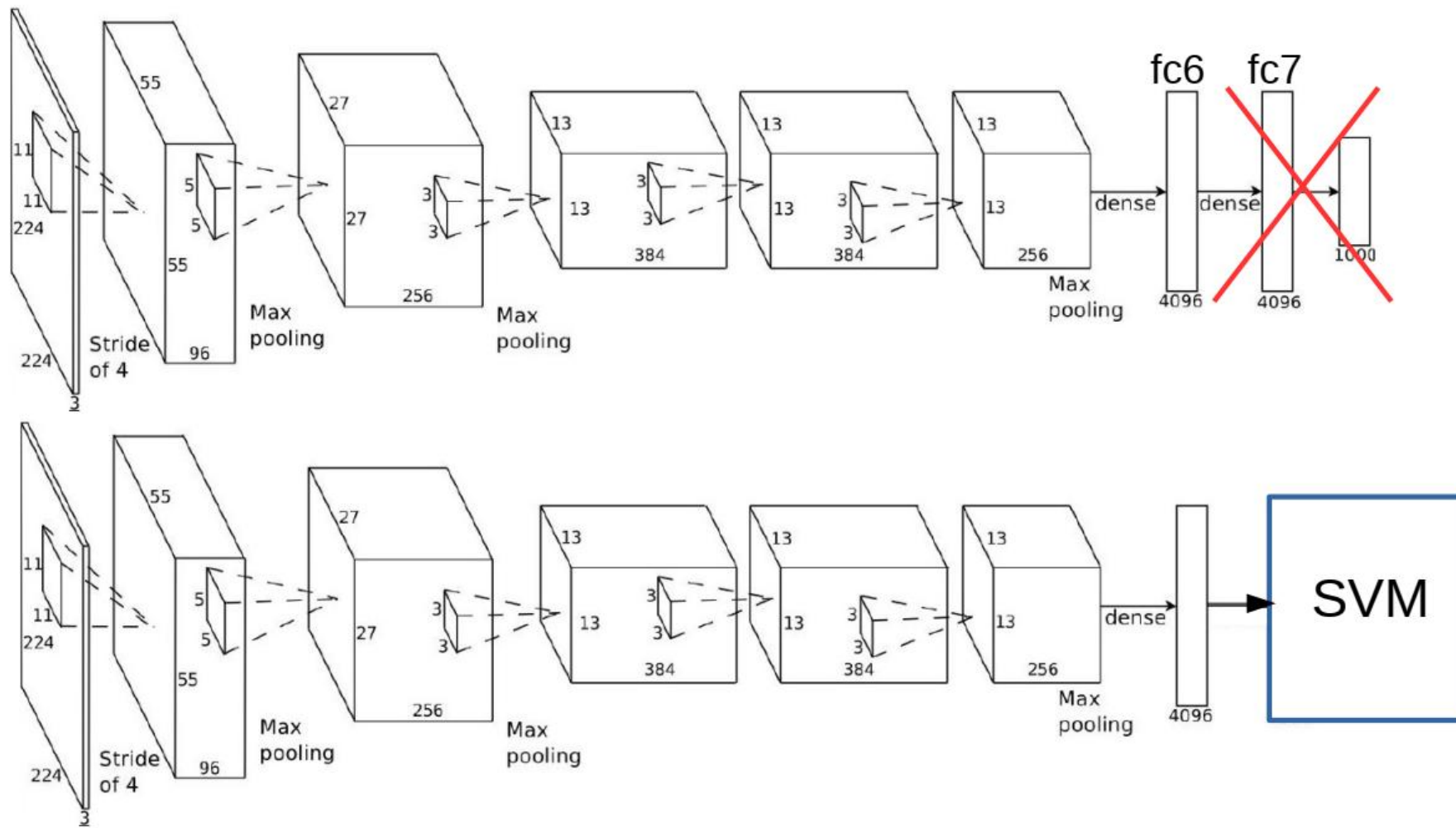
$$P_T(X_T) \approx P_S(X_S)$$

A nivel básico, existen dos mecanismos típicos para hacer esta adaptación

- **Transferencia directa:** *features* aprendidas en un dominio se utilizan directamente en otro, sin noción de la existencia de una red neuronal subyacente.
- **Fine-tuning:** las features aprendidas son reentrenadas “suavemente” en el nuevo dominio, cambiando el clasificador/regresor.

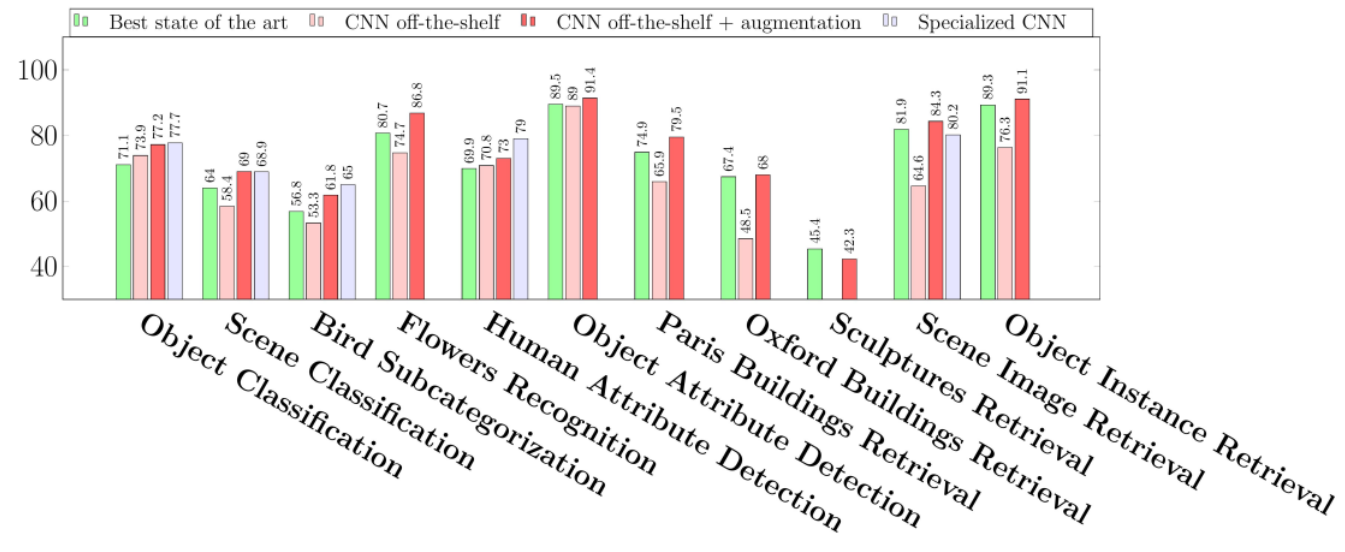
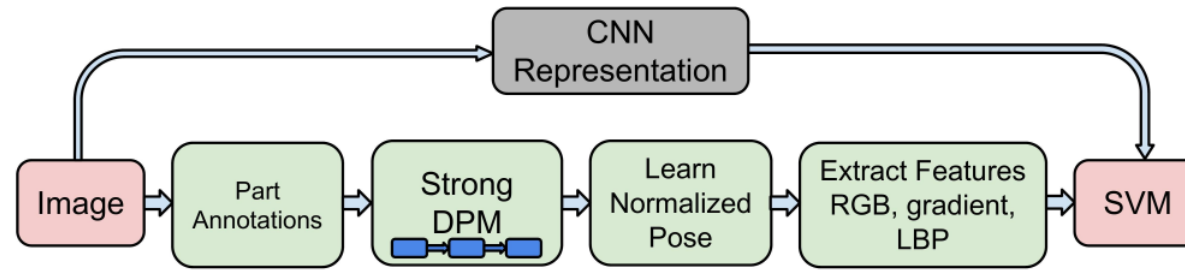


Transferencia directa de features

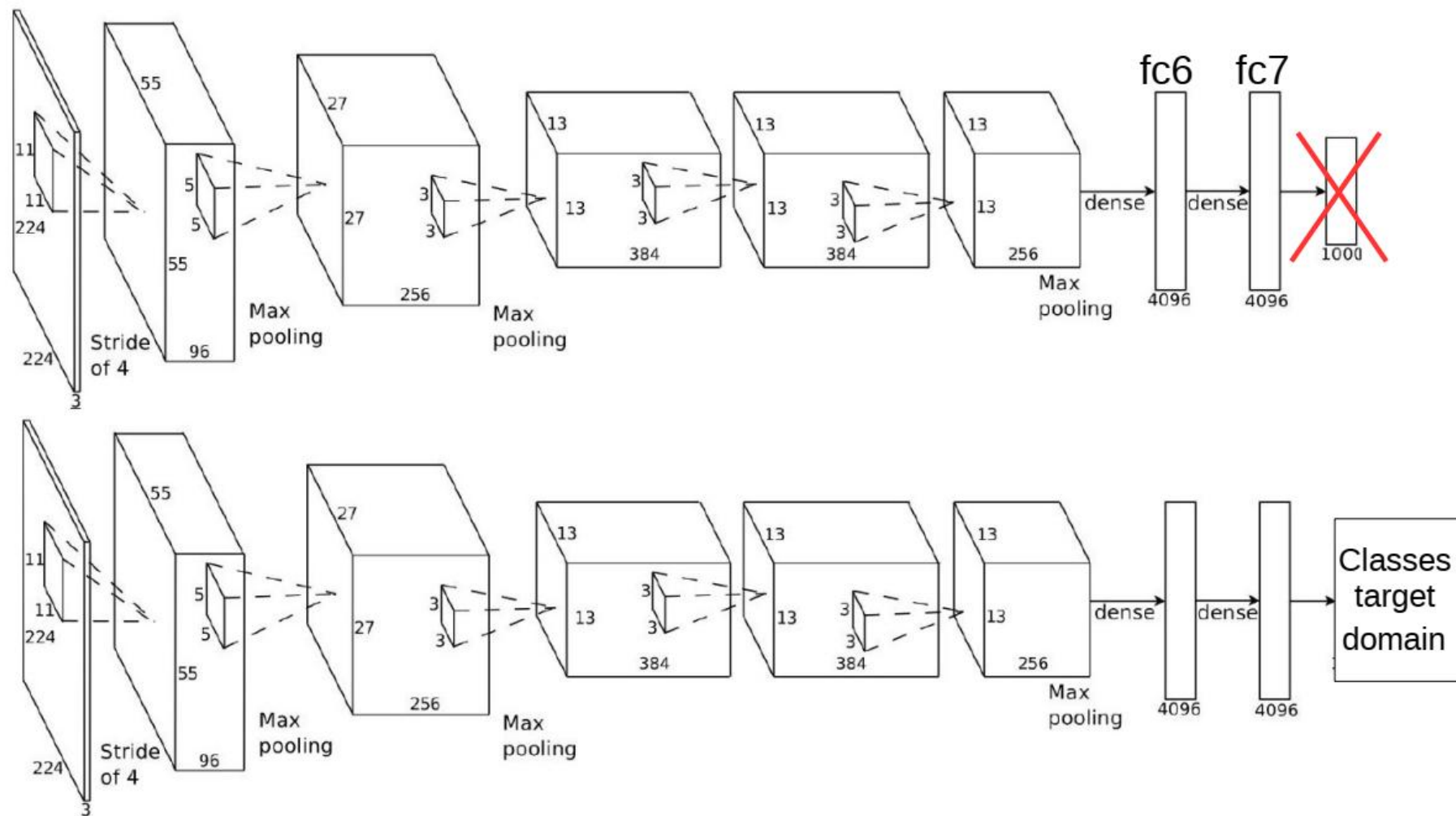


Transferencia directa de features

Ex. A. Razavian et al., “*CNN Features off-the-shelf: an Astounding Baseline for Recognition*”, 2014.

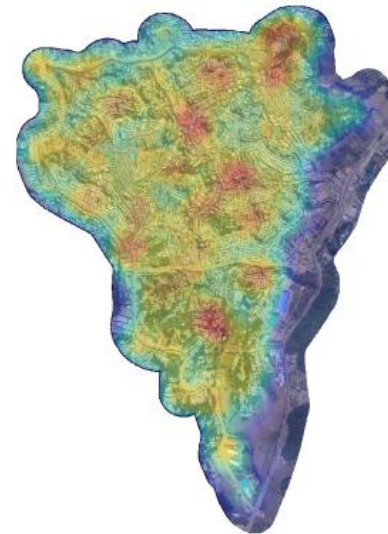
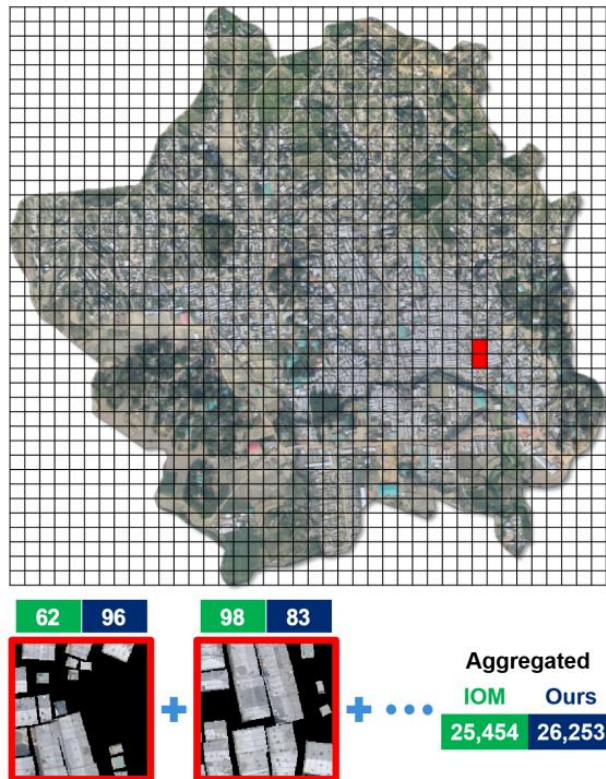


Fine-tuning de *features*

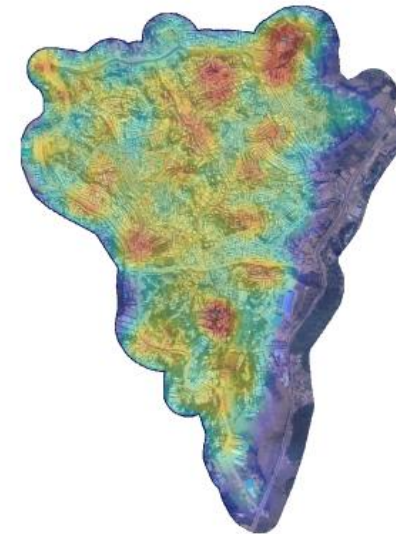


Fine-tuning de features

Estimating Displaced Populations From Overhead: Usando una CNN entrenada en ImageNet como base (ResNet-50), se cambia clasificador final por regresor y se reentrena usando imágenes satelitales.



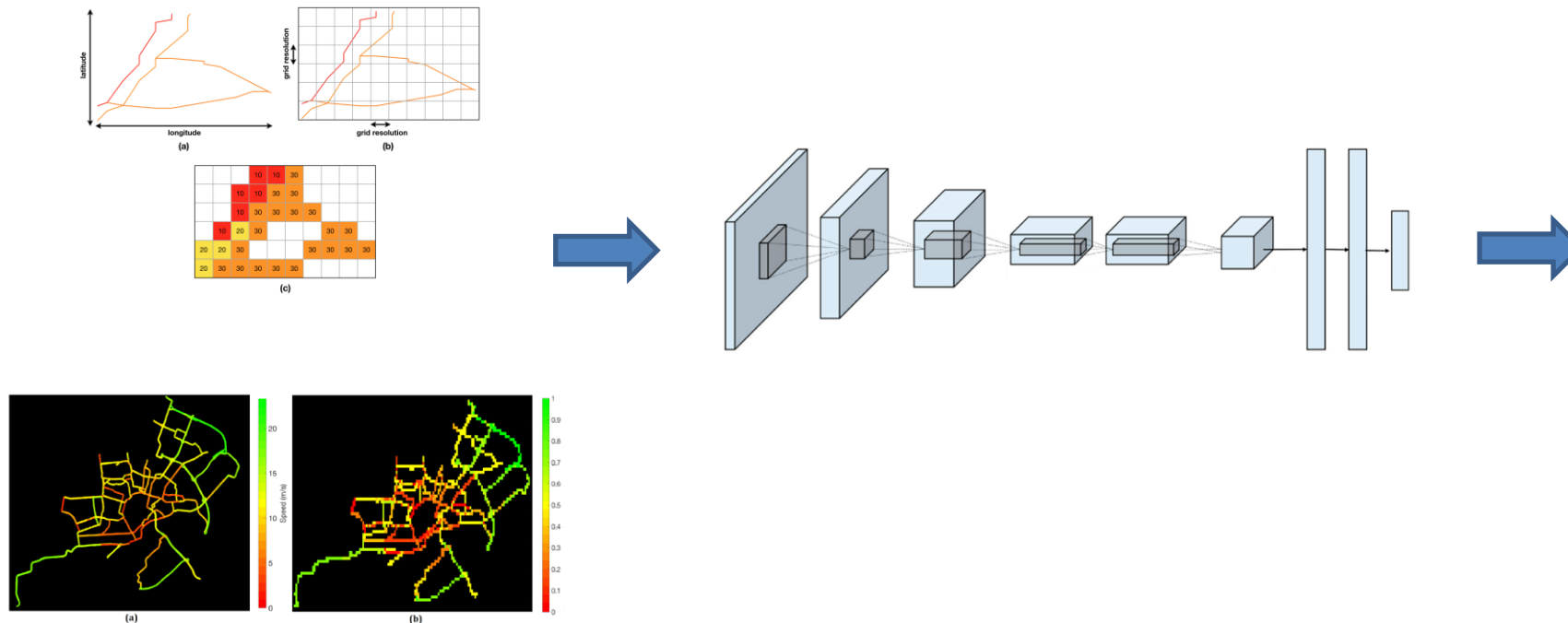
(a) Our Prediction



(b) IOM Estimate

Podemos ir más lejos aún

Understanding Network Traffic States using Transfer Learning: se usa CNN entrenada en ImageNet como base para extraer features de estados de tráfico codificados como imágenes y luego agruparlos.



Class	Medoid	Distribution of time slices in each class	Distribution of days in each class
1			
2			
3			
4			
5			

Transfer Learning presenta una gran cantidad de desafíos

- ¿Reentrenamos todas las capas o algunas? ¿Cuáles? ¿En qué orden?
- ¿Cuándo decidir si se hace fine-tuning o entrenamiento desde cero de una red?
- ¿Cuán similares son los dominios?
- ¿Qué rol juega el *learning rate*?

Cuál es EL CONCEPTO CENTRAL de esta clase

- Para entrenar redes hay que considerar muchos elementos prácticos
- Afortunadamente, no es necesario reinventar la rueda y muchas veces es suficiente basarse trabajos anteriores.

Antes de terminar

- El próximo martes tendremos el Control 1 durante aprox. las primeras 2 horas de la sesión. Este cubrirá la materia vista hasta hoy.
- Durante los próximos días se publicará el enunciado del Tarea 1, que cubrirá los aspectos prácticos (programación y análisis) de la materia vista hasta ahora y una guía de preparación para el control.
- Luego del control del próximo martes tendremos una sesión de consultas. La entrega de la Tarea 1 es el domingo 6 de septiembre.
- La sesión del próximo martes es remota.

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



INF3812 – Deep Learning

Entrenamiento de Redes Neuronales

Hans Löbel

Dpto. Ingeniería de Transporte y Logística
Dpto. Ciencia de la Computación